ESGI Lyon
5ESGI AL + IABD, année 2026-2027

Clusterisation de conteneurs

Rapport des travaux dirigés

Docker, Docker Compose, Docker Swarm, Traefik, Portainer,
Dozzle, Authelia, Outline

Prénom NOM

Novembre 2026

Sommaire

Présentation **4**

TD 1 : Dockerisation de My Favorite Places **5**

1. Test initial avec Bruno . . . 5
1.1 Lancer une base PostgreSQL . . . 5
1.2 Lancer l'API en mode développement . . . 5
1.3 Tests avec la collection Bruno . . . 5
1.3.a Créer un utilisateur . . . 5
1.3.b Se connecter . . . 6
1.3.c Consulter mon profil . . . 6
2. Dockerisation du serveur . . . 6
2.1 Le Dockerfile du serveur . . . 6
2.2 Adapter la connexion à la base . . . 6
2.3 Lancer la base et le serveur . . . 7
2.4 Test avec Bruno . . . 8
3. Dockerisation du client . . . 8
3.1 Le Dockerfile du client . . . 8
3.2 Adapter l'adresse de l'API . . . 8
3.3 Lancer le client . . . 8
3.4 Test dans le navigateur . . . 9
4. Orchestration avec Docker Compose . . . 10
4.1 Le fichier compose.yml . . . 10
4.2 Lancer l'application . . . 10
4.3 Test . . . 11
4.4 Simplification de la connexion à la base . . . 11
5. Communication entre les services . . . 12
5.1 Entre la base et le serveur . . . 12
5.2 Entre le client et le serveur . . . 12
6. Démarrage, redémarrage et images légères . . . 12
6.1 Démarrer le serveur quand la base est prête . . . 12
6.2 Redémarrage automatique . . . 12
6.3 Alléger les images . . . 13
6.3.a Serveur . . . 13
6.3.b Client . . . 13
6.3.c Résultat . . . 14

TD 2 : Cluster Docker Swarm **15**

Prérequis : images publiées sur un registry . . . 15

0.1 Construire les images avec GitHub Actions . . . 15

0.2 Vérifier les images sur GHCR . . . 15

Exercice 2 : Création du cluster Docker Swarm . . . 16

2.1 Lancer les 4 machines . . . 16

2.2 Initialiser le cluster . . . 17

2.3 Faire rejoindre les nœuds . . . 17

2.4 Lister les nœuds du cluster . . . 17

Exercice 3 : Tests du cluster . . . 17

3.1 Créer une stack hello-world . . . 17

3.2 Copier la stack dans le manager . . . 18

3.3 Déployer la stack et observer le cluster . . . 18

3.3.a Relancer le cluster après un redémarrage du PC . . . 18

3.3.b Déployer la stack hello . . . 19

3.4 Faire varier la clause deploy . . . 20

3.4.a Sur le manager uniquement . . . 20

3.4.b Sur tous les nœuds (mode global) . . . 21

TD 3 : Traefik et Portainer **23**

Exercice 1 : Déployer Traefik . . . 23

1.1 Relancer le cluster . . . 23

1.2 Déployer Traefik sur le manager . . . 23

1.3 Accéder à Traefik depuis le navigateur . . . 24

Exercice 2 : Déployer une autre application . . . 25

2.1 Tester l'application avec Docker Compose . . . 25

2.2 Déployer l'application sur le cluster Swarm . . . 26

2.3 Publier le vote et les résultats avec Traefik . . . 26

2.3.a Labels et redéploiement . . . 26

2.3.b Vérification . . . 27

Exercice 3 : Déployer Portainer . . . 28

3.1 Déployer Portainer sur le cluster . . . 28

3.2 Publier Portainer avec Traefik . . . 28

3.3 Ouvrir Portainer dans le navigateur . . . 29

3.3.a Setup token . . . 29

3.3.b Création du compte . . . 29

Exercice 4 : Redéployer depuis Portainer . . . 30

4.1 Supprimer la stack depuis le manager . . . 30

4.2 Republier la stack depuis Portainer . . . 30

4.3 Vérifier l'accès depuis le navigateur . . . 31

4.4 Déployer l'API de MFP depuis Portainer . . . 31

4.4.a Création de la stack . . . 31

4.4.b Test avec Bruno . . . 32

TD 4 : Secrets et configs Docker Swarm 33

Prérequis : relancer le cluster et passer en HTTPS . . . 33
0.1 Relancer le cluster . . . 33
0.2 Créer le certificat . . . 34
0.3 Le certificat dans le cluster : un secret et des configs . . . 34
0.4 Traefik en HTTPS . . . 35
0.5 Relancer Portainer et ajouter les adresses . . . 36

Exercice 1 : Déployer des logiciels . . . 37
1.1 Dozzle avec un utilisateur simple . . . 37
1.1.a Le fichier users.yml . . . 37
1.1.b Le secret, créé dans Portainer . . . 37
1.1.c La stack . . . 37
1.1.d Test dans le navigateur . . . 38
1.2 Authelia et connexion OpenID Connect . . . 38
1.2.a Les clés d'Authelia . . . 39
1.2.b Le compte dans un secret. . . . 39
1.2.c Le client OIDC de Dozzle . . . 39
1.2.d La configuration dans une config . . . 40
1.2.e Le déploiement . . . 40
1.2.f Brancher Dozzle sur Authelia . . . 41
1.2.g Rôles manquants et mise à jour de la config . . . 42
1.2.h Résultat . . . 43
1.3 Bonus : Outline . . . 44
1.3.a Les secrets d'Outline . . . 44
1.3.b Un nouveau client dans Authelia . . . 45
1.3.c La stack . . . 46
1.3.d Résultat . . . 46

Bilan. . . . 47

Présentation

Ce rapport regroupe les travaux dirigés du module. J'y ai repris, pour chaque question, les commandes que j'ai lancées et les captures du résultat obtenu.

Dépôt. Tout le code et les fichiers de configuration se trouvent sur `https://github.com/DARK-SHAD0W/Clusterisation-Conteneurs`.

- `mfp/` : l'application My Favorite Places (client React, API Node.js/Express, base PostgreSQL) et le cluster Docker Swarm (`mfp/swarm/`) ;
- `voting-app/` : l'application de vote utilisée au TD 3.

Environnement de travail.

Outil	Version	Remarque
Docker Engine	29.7.2	dans WSL 2 (Ubuntu 24.04), sans Docker Desktop
Docker Compose	v5.5.0	plugin `docker compose`
Docker des nœuds du cluster	29.8.2	image `docker:dind`
Node.js / Yarn	22.23.2 / 1.22.22	dans WSL
Bruno	4.2.1	client API, sous Windows

Toutes les commandes se lancent dans un terminal WSL. Celles du cluster se lancent depuis le dossier `mfp/swarm/`, où se trouve son fichier `compose.yml`.

Lecture. Chaque TD forme un chapitre. Les exercices gardent la numérotation de l'énoncé : la question 2 de l'exercice 1 est numérotée 1.2, et ses étapes 1.2.a, 1.2.b, etc. Le sommaire est cliquable, et le lien « Sommaire » en haut de chaque page permet d'y revenir.

TD 1 : Dockerisation de My Favorite Places

My Favorite Places est composée d'un client React, d'une API Node.js/Express et d'une base PostgreSQL. Dans ce TD, j'ai mis ces trois parties dans des conteneurs, puis je les ai orchestrées avec Docker Compose.

1. Test initial avec Bruno

1.1 Lancer une base PostgreSQL

Le serveur ne démarre pas sans base. J'ai lancé PostgreSQL dans un conteneur avec le mot de passe attendu par `server/src/datasource.ts`.

Commandes

```
docker run -d --name mfp-db \
  -e POSTGRES_PASSWORD=supersecret \
  -p 5432:5432 \
  postgres:17
docker ps
```

1.2 Lancer l'API en mode développement

Commandes

```
cd server
yarn install
yarn dev
```

La console affiche `connected to PgSQL db` puis `server started on port 3000`.

1.3 Tests avec la collection Bruno

J'ai ouvert le dossier `mfp-bruno-api-test-collection/` dans Bruno et lancé les requêtes dans l'ordre, avec le compte `ahmedtest@gmail.com` et le mot de passe `azerty123`.

1.3.a Créer un utilisateur

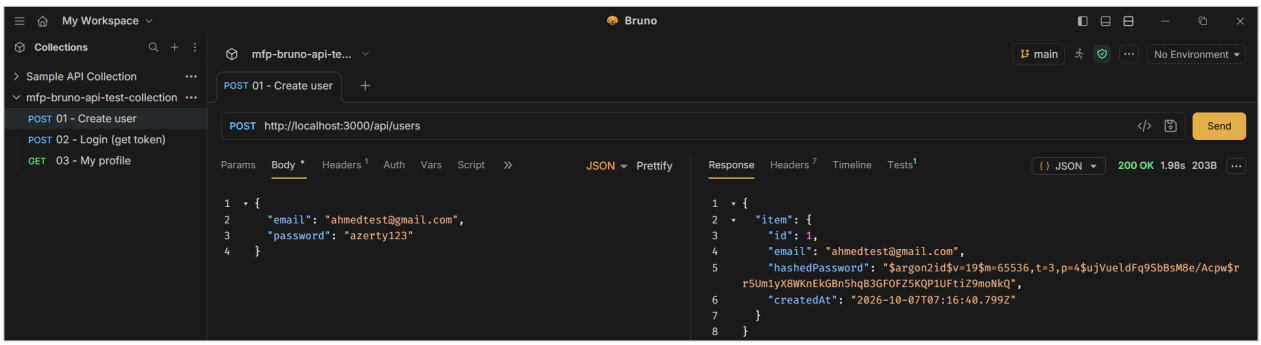


Figure 1.1. Requête « Create user » sur `POST /api/users` : l'API répond `200 OK` et renvoie l'utilisateur créé, avec son mot de passe haché.

1.3.b Se connecter

La requête « Login (get token) » sur `POST /api/users/tokens` renvoie `200 OK` et un token. La même requête est montrée en 2.4, avec le serveur dans Docker.

1.3.c Consulter mon profil

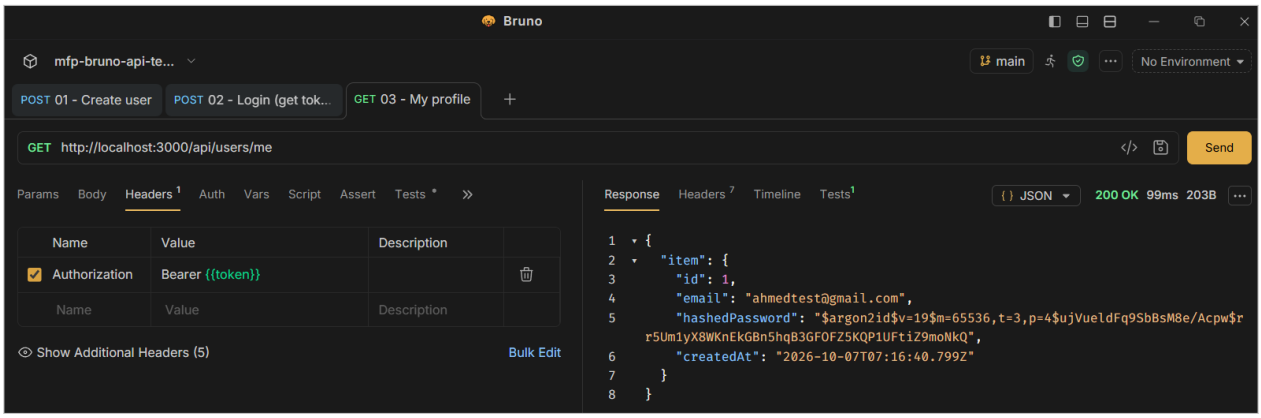


Figure 1.2. Requête « My profile » sur `GET /api/users/me`, avec le token dans l'en-tête `Authorization`. L'API répond `200 OK` et renvoie l'utilisateur connecté.

2. Dockerisation du serveur

2.1 Le Dockerfile du serveur

server/Dockerfile

```
FROM node:24-alpine3.24
WORKDIR /app
COPY package.json yarn.lock ./
RUN yarn install --frozen-lockfile
COPY tsconfig.json ./
COPY src src
RUN yarn build
EXPOSE 3000
CMD ["yarn", "start"]
```

J'ai aussi ajouté un `server/.dockerignore` qui écarte `node_modules` et `dist`. Les captures de ce TD ont été prises avec l'image `node:22`, que j'ai remplacée ensuite par `node:24-alpine3.24`.

2.2 Adapter la connexion à la base

Dans un conteneur, `localhost` désigne le conteneur lui-même. J'ai donc fait lire les paramètres de connexion dans des variables d'environnement.

server/src/datasource.ts

```
host: process.env.DB_HOST || "localhost",
username: process.env.DB_USER || "postgres",
password: process.env.DB_PASSWORD || "supersecret",
database: process.env.DB_NAME || "postgres",
```

2.3 Lancer la base et le serveur

J'ai créé un réseau commun, sur lequel le serveur trouve la base sous le nom `mfp-db`.

Commandes

```
# 1. créer un réseau commun
docker network create mfp-net
# 2. lancer la base sur ce réseau
docker run -d --name mfp-db --network mfp-net \
  -e POSTGRES_PASSWORD=supersecret \
  postgres:17
# 3. construire l'image du serveur
docker build -t mfp-server ./server
# 4. lancer le serveur sur le même réseau
docker run -d --name mfp-server --network mfp-net \
  -e DB_HOST=mfp-db \
  -p 3000:3000 \
  mfp-server
```

Problème rencontré : La commande 4 a échoué avec `address already in use` sur le port 3000, car le `yarn dev` de la partie 1 tournait encore.

Solution : J'ai arrêté `yarn dev` avec Ctrl+C, supprimé le conteneur raté et relancé la commande 4.

Commandes

```
docker rm -f mfp-server
# 4. lancer le serveur sur le même réseau
docker run -d --name mfp-server --network mfp-net \
  -e DB_HOST=mfp-db \
  -p 3000:3000 \
  mfp-server
# 5. vérifier
docker ps
docker logs mfp-server
```

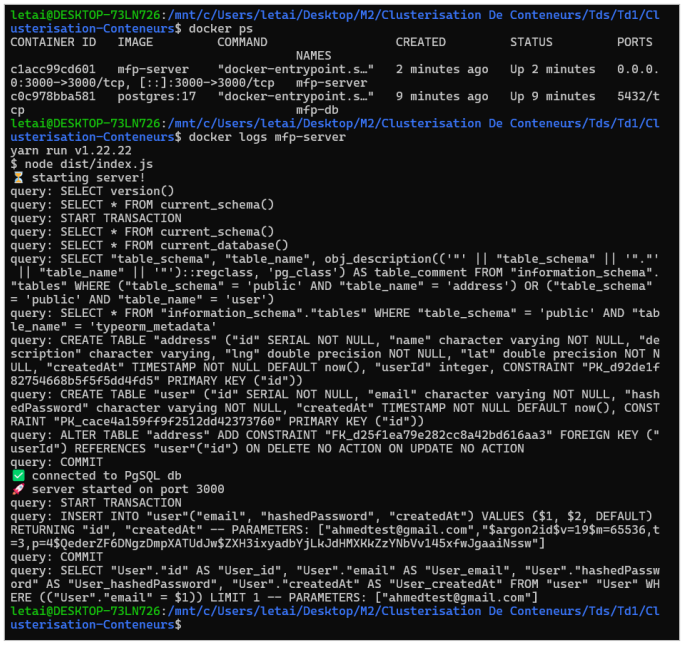


Figure 1.3. Les conteneurs `mfp-server` et `mfp-db` sont `Up`. Dans les logs, le serveur crée les tables `user` et `address`, puis se connecte à la base.

2.4 Test avec Bruno

Grâce à `-p 3000:3000`, l'API répond à la même adresse qu'avec `yarn dev`. Comme la base était neuve, j'ai d'abord recréé l'utilisateur avec « Create user ».

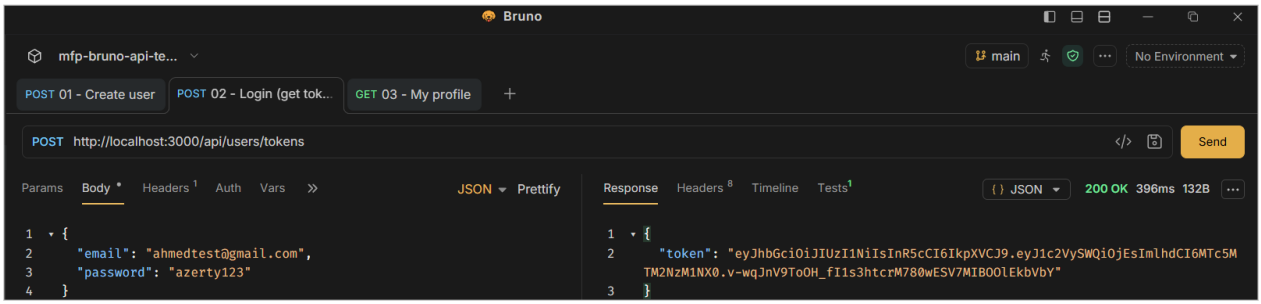


Figure 1.4. Requête « Login (get token) » vers le serveur lancé dans Docker : `200 OK` et un token. Le serveur retrouve bien l'utilisateur dans `mfp-db`.

3. Dockerisation du client

3.1 Le Dockerfile du client

client/Dockerfile

```
FROM node:24-alpine3.24
WORKDIR /app
COPY package*.json yarn.lock ./
RUN yarn install --frozen-lockfile
COPY tsconfig*.json ./
COPY vite.config.ts ./
COPY index.html ./
COPY public public
COPY src src
EXPOSE 5173
CMD ["yarn", "dev", "--host"]
```

Sans l'option `--host`, Vite n'accepte que les connexions venant de l'intérieur du conteneur.

3.2 Adapter l'adresse de l'API

Vite fait suivre les appels `/api` au serveur. Comme pour la base, j'ai mis l'adresse du serveur dans une variable d'environnement.

client/vite.config.ts

```
target: process.env.API_URL || "http://localhost:3000",
```

3.3 Lancer le client

La base et le serveur de la partie 2 tournent toujours.

Commandes

```
# 1. construire l'image du client
docker build -t mfp-client ./client
# 2. lancer le client sur le même réseau que le serveur
docker run -d --name mfp-client --network mfp-net \
```

```
  -e API_URL=http://mfp-server:3000 \
  -p 5173:5173 \
  mfp-client
# 3. vérifier
docker ps
docker logs mfp-client
```

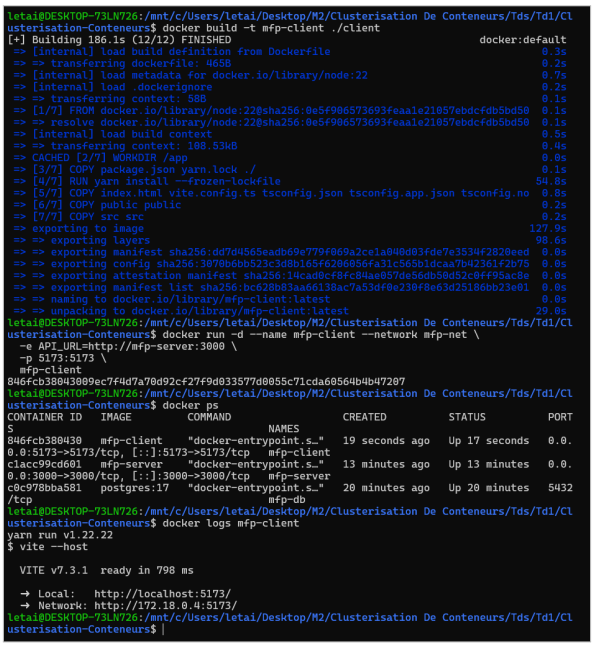


Figure 1.5. Les trois conteneurs `mfp-client`, `mfp-server` et `mfp-db` sont `Up`. Les logs du client montrent Vite prêt sur le port 5173.

3.4 Test dans le navigateur

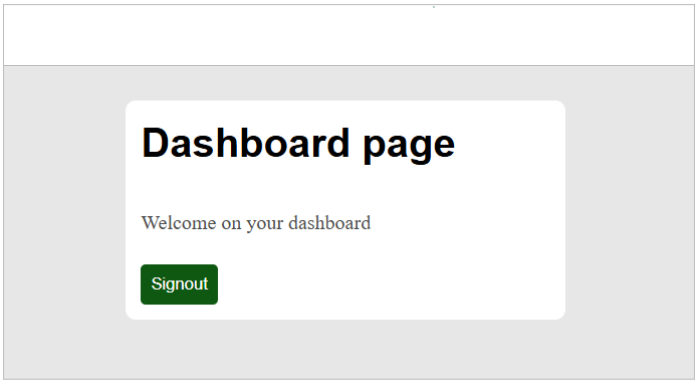


Figure 1.6. Connexion sur `http://localhost:5173` avec le compte de test : le tableau de bord s'affiche.

4. Orchestration avec Docker Compose

4.1 Le fichier compose.yml

Chaque service remplace une commande `docker run` que je tapais à la main. Compose crée lui-même le réseau, sur lequel chaque service est joignable par son nom.

compose.yml (extrait)

```yaml
services:
  db:
    image: postgres:17
    restart: unless-stopped
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: supersecret
    ports:
      - "5432:5432"
    volumes:
      - db-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 2s
      timeout: 5s
      retries: 15

  server:
    build: ./server
    restart: unless-stopped
    ports:
      - "3000:3000"
    depends_on:
      db:
        condition: service_healthy
```

Le service `client` publie le port 5173 et dépend de `server`. Le volume `db-data` garde les données de la base quand le conteneur est recréé.

4.2 Lancer l'application

J'ai d'abord supprimé les conteneurs et le réseau créés à la main, qui occupaient encore les ports 3000 et 5173.

Commandes

```
docker rm -f mfp-client mfp-server mfp-db
docker network rm mfp-net

docker compose up -d --build
docker compose ps
docker compose logs server
```

Problème rencontré : Avec un simple `depends_on`, le serveur s'arrêtait sur `ECONNREFUSED`, car PostgreSQL n'était pas encore prêt au démarrage du serveur.

Solution : J'ai ajouté le `healthcheck` sur `db` et `condition: service_healthy` sur `server`.

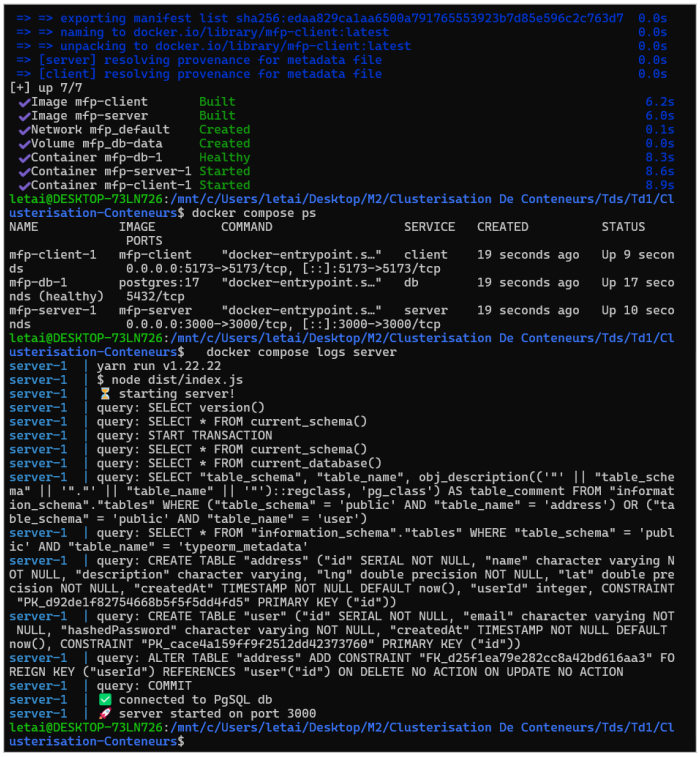


Figure 1.7. Compose attend que `mfp-db-1` soit `Healthy` avant de démarrer le serveur et le client. Les trois services sont `Up` et le serveur se connecte à la base.

4.3 Test

Le volume étant neuf, j'ai recréé l'utilisateur avec Bruno, puis je me suis connecté au client avec le même résultat qu'en 3.4. Pour tout arrêter sans perdre le volume `db-data` :

Commandes

```
docker compose down
```

4.4 Simplification de la connexion à la base

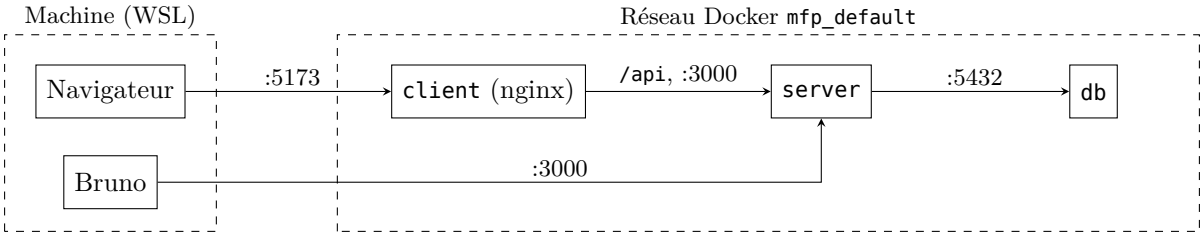
Avec Compose, la base s'appelle toujours `db`. J'ai donc retiré les variables d'environnement et écrit l'adresse directement dans le code.

server/src/datasource.ts

```
host: "db",
username: "postgres",
password: "supersecret",
database: "postgres",
```

En contrepartie, le serveur ne trouve plus la base en dehors de Compose.

5. Communication entre les services



5.1 Entre la base et le serveur

Les deux conteneurs sont sur le réseau `mfp_default` créé par Compose. Le serveur appelle la base par son nom de service, `db`, sur le port 5432, et Docker traduit ce nom en adresse IP. Le port 5432 n'est publié que pour consulter la base depuis ma machine.

5.2 Entre le client et le serveur

Le code React s'exécute dans le navigateur, hors du réseau Docker, et appelle donc l'API à l'adresse du site (`http://localhost:5173/api/...`). Le conteneur `client` relaie ces appels vers le service `server`, d'abord avec Vite puis avec nginx. Bruno, lui, appelle directement le serveur sur le port 3000 publié.

6. Démarrage, redémarrage et images légères

6.1 Démarrer le serveur quand la base est prête

C'était déjà réglé en 4.2 avec le `healthcheck` sur `db` et `condition: service_healthy` sur `server`.

6.2 Redémarrage automatique

J'ai ajouté cette ligne à chaque service de `compose.yml`.

compose.yml

```
restart: unless-stopped
```

Pour le vérifier, j'ai simulé un plantage du serveur, puis un arrêt manuel.

Commandes

```
docker compose exec server pkill -f "node dist/index.js"
docker inspect mfp-server-1 --format 'status={{.State.Status}} restarts={{.RestartCount}}'
# status=running restarts=1

docker compose stop server
docker inspect mfp-server-1 --format 'status={{.State.Status}} restarts={{.RestartCount}}'
# status=exited restarts=1   (arrêt manuel : pas de redémarrage)

docker compose start server
```

6.3 Alléger les images

J'ai découpé chaque Dockerfile en trois étapes : `deps` installe les dépendances, `builder` construit l'application et `main` ne garde que ce qui sert à la lancer.

6.3.a Serveur

L'image finale ne garde que le JavaScript compilé et les dépendances de production.

server/Dockerfile

```
FROM node:24-alpine3.24 AS deps
WORKDIR /app
COPY package.json yarn.lock ./
RUN yarn install --frozen-lockfile

FROM node:24-alpine3.24 AS builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY package.json ./
COPY src ./src
COPY tsconfig.json ./

RUN yarn build

FROM node:24-alpine3.24 AS main
WORKDIR /app

COPY --from=builder /app/dist ./dist
COPY package.json yarn.lock ./
RUN yarn install --frozen-lockfile --production && yarn cache clean

EXPOSE 3000

CMD ["yarn", "start"]
```

6.3.b Client

Une fois construit, le client n'est plus qu'un ensemble de fichiers statiques. Les étapes `deps` et `builder` sont les mêmes que pour le serveur, mais l'image finale part de nginx.

client/Dockerfile (étape finale)

```
# ---- stage 3: production image, nginx serves the static files ----
FROM nginx:1.29-alpine AS main

COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=builder /app/dist /usr/share/nginx/html

EXPOSE 5173
```

Dans cette image, nginx reprend le rôle de Vite : il fait suivre les appels `/api/` au service `server`.

client/nginx.conf

```
server {
    listen 5173;
    root /usr/share/nginx/html;

    # React routes (/signin, /dashboard...) all load index.html
```

```
    location / {
        try_files $uri $uri/ /index.html;
    }

    # API calls are forwarded to the server service
    location /api/ {
        proxy_pass http://server:3000;
    }
}
```

6.3.c Résultat

Commandes

```
docker compose up -d --build
docker images
```

Image	Avant	Après
`mfp-server`	477 Mo	337 Mo
`mfp-client`	1,49 Go	93 Mo

TD 2 : Cluster Docker Swarm

Dans ce TD, j'ai monté un cluster Docker Swarm local de 4 machines. Chaque machine est un conteneur Docker-in-Docker (`docker:dind`) : 1 manager et 3 nœuds.

Prérequis : images publiées sur un registry

0.1 Construire les images avec GitHub Actions

Swarm ne sait pas construire d'images : j'ai donc ajouté deux workflows qui publient le serveur et le client sur GHCR à chaque push sur `main`.

.github/workflows/build.server.yml

```yaml
name: Build and push server image

on:
  push:
    branches: ['main']
    paths:
      - 'mfp/server/**'
      - '.github/workflows/build.server.yml'
  workflow_dispatch:  # also allows a manual run from the Actions tab

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${{ github.repository }}/server
```

0.2 Vérifier les images sur GHCR

Les deux images sont publiques et téléchargeables sans connexion :

- `ghcr.io/dark-shad0w/clusterisation-conteneurs/server:latest`
- `ghcr.io/dark-shad0w/clusterisation-conteneurs/client:latest`

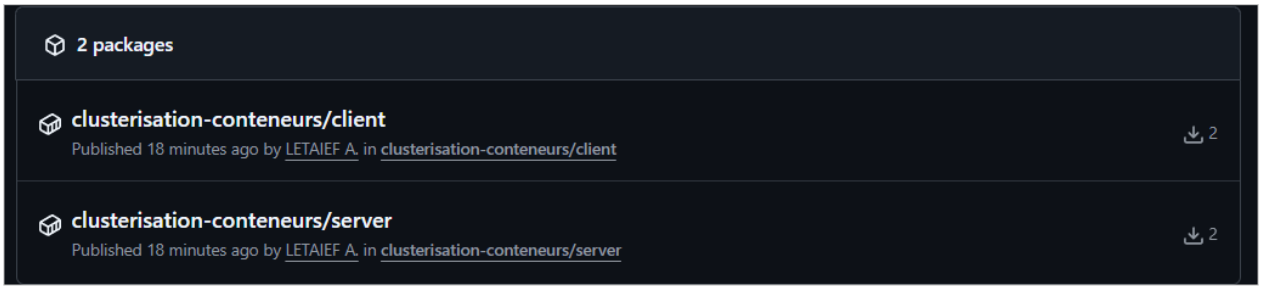


Figure 2.1. Les packages `clusterisation-conteneurs/client` et `clusterisation-conteneurs/server` publiés sur GHCR.

Exercice 2 : Création du cluster Docker Swarm

2.1 Lancer les 4 machines

Le fichier `swarm/compose.yml` crée le `manager` et les nœuds `node1` à `node3`, tous définis de la même façon.

swarm/compose.yml

```yaml
# Local Docker Swarm cluster: each "machine" is a Docker-in-Docker container
name: swarm

services:
  manager:
    image: docker:dind
    hostname: manager
    privileged: true

  node1:
    image: docker:dind
    hostname: node1
    privileged: true
```

Commandes

```
cd swarm
docker compose up -d
docker ps
```

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Cl
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Cl
usterisation-Conteneurs/swarm$  docker compose up -d
[+] up 22/22
 ✔Image docker:dind          Pulled                                                  29.5s
 ✔Network swarm_default      Created                                                  0.1s
 ✔Container swarm-node3-1    Started                                                  1.4s
 ✔Container swarm-manager-1  Started                                                  1.5s
 ✔Container swarm-node1-1    Started                                                  1.4s
 ✔Container swarm-node2-1    Started                                                  1.5s
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Cl
usterisation-Conteneurs/swarm$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED             STATUS
       PORTS                                                 NAMES
8db993eaa83c   docker:dind    "dockerd-entrypoint.…"   24 seconds ago      Up 22 seconds
       2375-2376/tcp                                         swarm-manager-1
12a3a3bf1b09   docker:dind    "dockerd-entrypoint.…"   24 seconds ago      Up 22 seconds
       2375-2376/tcp                                         swarm-node2-1
aacec484287b   docker:dind    "dockerd-entrypoint.…"   24 seconds ago      Up 22 seconds
       2375-2376/tcp                                         swarm-node3-1
57aa55d79f91   docker:dind    "dockerd-entrypoint.…"   24 seconds ago      Up 22 seconds
       2375-2376/tcp                                         swarm-node1-1
11ed06c1a8a4   mfp-client     "/docker-entrypoint.…"   About an hour ago   Up About an hour
       80/tcp, 0.0.0.0:5173->5173/tcp, [::]:5173->5173/tcp   mfp-client-1
f6b1980ab4cb   c9ab01022362   "docker-entrypoint.s…"   2 hours ago         Up About an hour
       0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp           mfp-server-1
4c4030c6d31d   postgres:17    "docker-entrypoint.s…"   2 hours ago         Up 2 hours (heal
thy)   0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp           mfp-db-1
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Cl
```

Figure 2.2. Les 4 conteneurs `swarm-manager-1`, `swarm-node1-1`, `swarm-node2-1` et `swarm-node3-1` sont `Up`, à côté des conteneurs du TD 1.

J'ai vérifié que Docker répond à l'intérieur : la liste est vide, chaque machine a son propre Docker.

Commandes

```
docker compose exec manager docker ps
docker compose exec node1 docker ps
```

2.2 Initialiser le cluster

Commandes

```
docker compose exec manager docker swarm init
```

Le conteneur `manager` devient le manager du cluster et Docker affiche le token `SWMTKN-1-...` à donner aux nœuds.

2.3 Faire rejoindre les nœuds

J'ai rangé le token dans une variable, puis chaque nœud a rejoint le manager par son nom de service.

Commandes

```
TOKEN=$(docker compose exec manager docker swarm join-token -q worker)
docker compose exec node1 docker swarm join --token $TOKEN manager:2377
docker compose exec node2 docker swarm join --token $TOKEN manager:2377
docker compose exec node3 docker swarm join --token $TOKEN manager:2377
```

2.4 Lister les nœuds du cluster

Je suis entré dans le conteneur `manager` pour lui demander la liste des nœuds.

Commandes

```
docker compose exec manager ash
docker node ls
exit
```

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusteris
ation-Conteneurs/swarm$  docker compose exec manager ash
/ #  docker node ls
ID                            HOSTNAME   STATUS    AVAILABILITY   MANAGER STATUS   ENGINE VERSION
ut2j0ctz6094h5y236qwnbii5 *   manager    Ready     Active         Leader           29.8.2
gkqoimullj55ae70vubvjpq3t     node1      Ready     Active                          29.8.2
htlyy0s49t7h787sznf9syr0h     node2      Ready     Active                          29.8.2
tih4g88m7eipgra6a02nwvi17     node3      Ready     Active                          29.8.2
/ # exit
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusteris
ation-Conteneurs/swarm$
```

Figure 2.3. Les 4 nœuds sont `Ready` et `Active`. Le `manager` est `Leader` et les trois autres nœuds sont des workers.

Exercice 3 : Tests du cluster

3.1 Créer une stack hello-world

La stack demande au cluster 2 copies du conteneur `hello-world`.

swarm/hello-world.compose.yml

```yaml
services:
  hello:
    image: hello-world
    deploy:
      replicas: 2
```

3.2 Copier la stack dans le manager

J'ai ajouté un volume anonyme sur `/home/manager` pour que les fichiers copiés survivent à une recréation du conteneur.

swarm/compose.yml

```yaml
manager:
  image: docker:dind
  hostname: manager
  privileged: true
  volumes:
    - /home/manager
```

J'ai recréé le manager, vérifié que les 4 nœuds étaient toujours `Ready`, puis copié la stack dedans.

Commandes

```
docker compose up -d manager
docker compose exec manager docker node ls
docker compose exec manager mkdir -p /home/manager
docker compose cp hello-world.compose.yml manager:/home/manager/
docker compose exec manager ls /home/manager
```

3.3 Déployer la stack et observer le cluster

3.3.a Relancer le cluster après un redémarrage du PC

Après un redémarrage du PC, j'ai relancé les 4 conteneurs.

Commandes

```
docker compose start
docker compose exec manager docker node ls
```

Problème rencontré : Les nœuds `node1`, `node2` et `node3` sont restés en `Unknown` puis `Down`, car le manager avait changé d'adresse IP (de `172.19.0.5` à `172.19.0.2`).

Solution : J'ai fait quitter le cluster à toutes les machines, puis je l'ai recréé avec les commandes des questions 2.2 et 2.3.

Commandes

```
docker compose exec node1 docker swarm leave --force
docker compose exec node2 docker swarm leave --force
docker compose exec node3 docker swarm leave --force
docker compose exec manager docker swarm leave --force

docker compose exec manager docker swarm init
TOKEN=$(docker compose exec manager docker swarm join-token -q worker)
docker compose exec node1 docker swarm join --token $TOKEN manager:2377
docker compose exec node2 docker swarm join --token $TOKEN manager:2377
docker compose exec node3 docker swarm join --token $TOKEN manager:2377

docker compose exec manager docker node ls
```

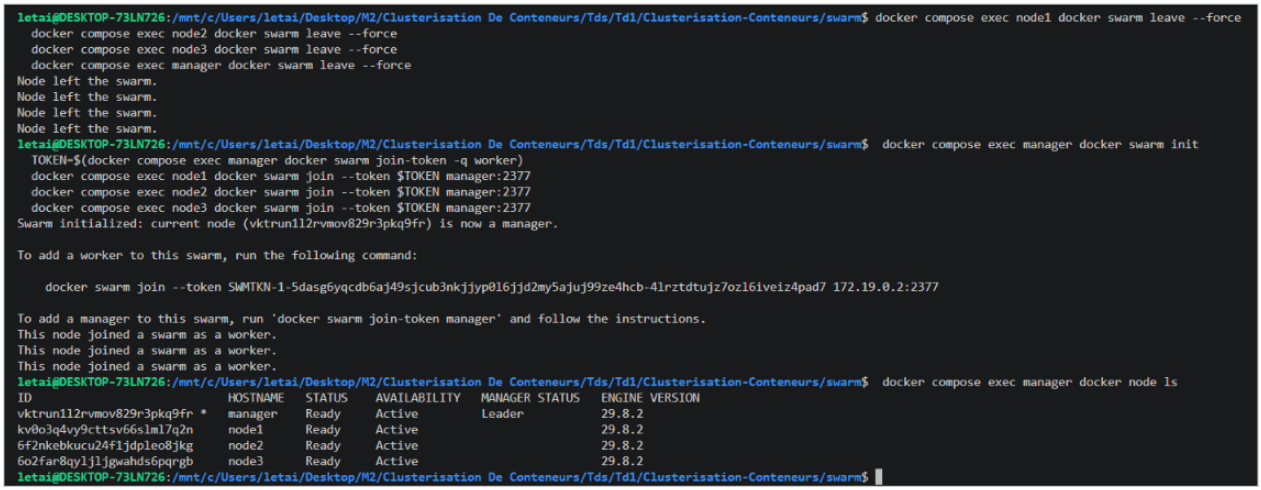


Figure 2.4. Après la recréation du cluster, les 4 nœuds sont de nouveau `Ready` et le `manager` a repris sa place de `Leader`.

3.3.b Déployer la stack hello

J'ai déployé la stack sous le nom `hello` depuis le manager.

Commandes

```
docker compose exec manager docker stack deploy -c /home/manager/hello-world.compose.yml
↪  hello
docker compose exec manager docker service ls
docker compose exec manager docker service ps hello_hello
```

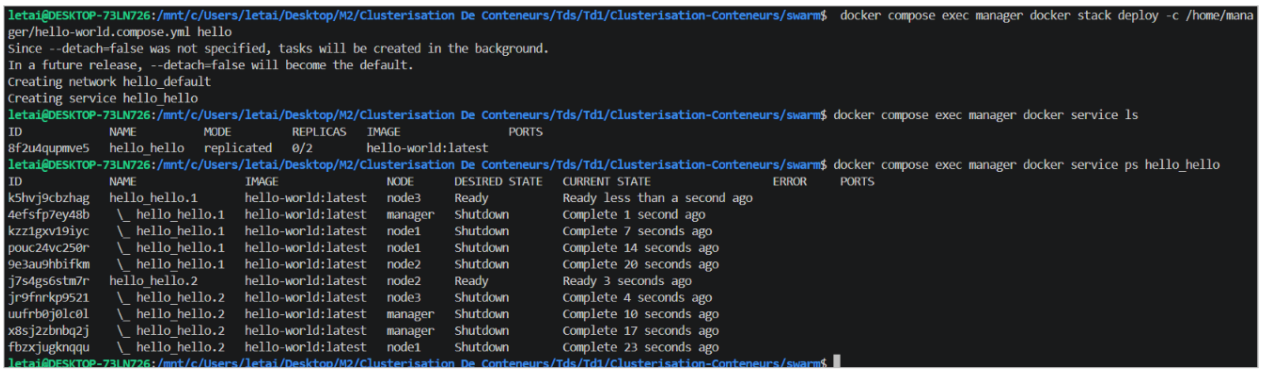


Figure 2.5. Le service `hello_hello` est à `0/2` car chaque copie affiche son message puis se termine (`Complete`). Swarm relance les copies sur des nœuds différents, `manager` compris.

Pour comparer, j'ai lancé un service nginx de 3 copies qui reste allumé : il affiche `3/3`, puis je l'ai supprimé.

Commandes

```
docker compose exec manager docker service create --name web --replicas 3 nginx:alpine
docker compose exec manager docker service ls
docker compose exec manager docker service rm web
```

J'ai ensuite regardé les conteneurs de chaque machine, arrêtés compris.

Commandes

```
docker compose exec manager docker ps -a
docker compose exec node1 docker ps -a
docker compose exec node2 docker ps -a
docker compose exec node3 docker ps -a
```

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$  docker compose exec manager do
cker ps -a
CONTAINER ID   IMAGE              COMMAND    CREATED          STATUS                      PORTS     NAMES
f355638693eb   hello-world:latest   "/hello"   9 seconds ago    Exited (0) 2 seconds ago              hello_hello.1.xsi3ohviq3j43lx5xuadtdrvw
f24aef73124b   hello-world:latest   "/hello"   16 seconds ago   Exited (0) 10 seconds ago             hello_hello.1.7sigjlu29gbypa3hod40cx39m
bfbf0de6f4ad   hello-world:latest   "/hello"   24 seconds ago   Exited (0) 17 seconds ago             hello_hello.2.y2xt2fd35e37cbadcod13i7pa
0833d06d9755   hello-world:latest   "/hello"   31 seconds ago   Exited (0) 24 seconds ago             hello_hello.2.5a31wbqdf4gqphyevylmc2vky
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$ docker compose exec node1 docke
r ps -a
CONTAINER ID   IMAGE              COMMAND    CREATED          STATUS                      PORTS     NAMES
b12bf34dff67   hello-world:latest   "/hello"   2 seconds ago    Created                             hello_hello.2.noviml5uf5s40265yxrk5806n
07d1bfb7b503   hello-world:latest   "/hello"   27 seconds ago   Exited (0) 22 seconds ago           hello_hello.2.6uho387zyzllzoeie4jiux84v
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$ docker compose exec node2 docke
r ps -a
CONTAINER ID   IMAGE              COMMAND    CREATED          STATUS                      PORTS     NAMES
939973e82e41   hello-world:latest   "/hello"   22 seconds ago   Exited (0) 16 seconds ago           hello_hello.1.0fmw0lejau2izpjzruozpsw7v
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$ docker compose exec node3 docke
r ps -a
CONTAINER ID   IMAGE              COMMAND    CREATED          STATUS                      PORTS     NAMES
ada9e02e6213   hello-world:latest   "/hello"   23 seconds ago   Exited (0) 17 seconds ago           hello_hello.1.xg3wkzwk1r2udwnx33bxxaqxb
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$
```

Figure 2.6. Chaque machine ne liste que ses propres copies de `hello-world`, la plupart déjà en `Exited (0)`. Swarm en a placé sur les 4 machines, `manager` compris.

3.4 Faire varier la clause deploy

Pour chaque essai, j'ai supprimé la stack, copié le nouveau fichier dans le manager et redéployé.

3.4.a Sur le manager uniquement

swarm/hello-world-manager.compose.yml

```yaml
services:
  hello:
    image: hello-world
    deploy:
      replicas: 2
      placement:
        constraints:
          - node.role == manager
```

Commandes

```
docker compose exec manager docker stack rm hello
docker compose cp hello-world-manager.compose.yml manager:/home/manager/
docker compose exec manager docker stack deploy -c
↪  /home/manager/hello-world-manager.compose.yml hello
docker compose exec manager docker service ps hello_hello
```

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$
docker compose exec manager docker stack rm hello
Removing service hello_hello
Removing network hello_default
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$
  docker compose cp hello-world-manager.compose.yml manager:/home/manager/
[+] copy 1/1
 ✓ swarm-manager-1 Copied hello-world-manager.compose.yml to swarm-manager-1:/home/manager/                    0.1s
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$
 docker compose exec manager docker stack deploy -c /home/manager/hello-world-manager.compose.yml hello
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network hello_default
Creating service hello_hello
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$
 docker compose exec manager docker service ps hello_hello
ID             NAME                IMAGE                NODE      DESIRED STATE   CURRENT STATE                ERROR
 PORTS
zfjhmrzuo4fz   hello_hello.1       hello-world:latest   manager   Ready           Ready less than a second ago

0z7vmg3rvvnu    \_ hello_hello.1   hello-world:latest   manager   Shutdown        Complete 1 second ago

0s8l81lgrk1a    \_ hello_hello.1   hello-world:latest   manager   Shutdown        Complete 7 seconds ago

qkg5g80puhls   hello_hello.2       hello-world:latest   manager   Ready           Ready less than a second ago

lod0t1rgd1ya    \_ hello_hello.2   hello-world:latest   manager   Shutdown        Complete 1 second ago

vq2ejxa6qsuo    \_ hello_hello.2   hello-world:latest   manager   Shutdown        Complete 7 seconds ago

letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$
```

Figure 2.7. Avec la contrainte `node.role == manager`, toutes les copies tournent sur le `manager` et aucune sur les autres nœuds.

3.4.b Sur tous les nœuds (mode global)

swarm/hello-world-global.compose.yml

```yaml
services:
  hello:
    image: hello-world
    deploy:
      mode: global
```

Commandes

```
docker compose exec manager docker stack rm hello
docker compose cp hello-world-global.compose.yml manager:/home/manager/
docker compose exec manager docker stack deploy -c
↪  /home/manager/hello-world-global.compose.yml hello
docker compose exec manager docker service ls
docker compose exec manager docker service ps hello_hello
```

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$ docker compose exec manager do
cker stack rm hello
Removing service hello_hello
Removing network hello_default
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$ docker compose cp hello-world-
global.compose.yml manager:/home/manager/
[+] copy 1/1
 ✓ swarm-manager-1 Copied hello-world-global.compose.yml to swarm-manager-1:/home/manager/                                                    0.1s
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$ docker compose exec manager doc
ker stack deploy -c /home/manager/hello-world-global.compose.yml hello
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network hello_default
Creating service hello_hello
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$ docker compose exec manager do
cker service ls
ID             NAME          MODE      REPLICAS   IMAGE               PORTS
rno40lm6b9zl   hello_hello   global    0/0        hello-world:latest
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$ docker compose exec manager do
cker service ps hello_hello
ID             NAME                                     IMAGE               NODE      DESIRED STATE   CURRENT STATE                ERROR     PORTS
p16kq27gcitt   hello_hello.6f2nkebkucu24f1jdpleo8jkg     hello-world:latest   node2     Ready           Ready less than a second ago
spzm3npdcc0n    \_ hello_hello.6f2nkebkucu24f1jdpleo8jkg   hello-world:latest   node2     Shutdown        Complete 1 second ago
fbfe974s5a7d    \_ hello_hello.6f2nkebkucu24f1jdpleo8jkg   hello-world:latest   node2     Shutdown        Complete 9 seconds ago
kfi1wj4o3psc    \_ hello_hello.6f2nkebkucu24f1jdpleo8jkg   hello-world:latest   node2     Shutdown        Complete 15 seconds ago
ozlzlcgou9mc    \_ hello_hello.6f2nkebkucu24f1jdpleo8jkg   hello-world:latest   node2     Shutdown        Complete 22 seconds ago
74fck10ko4cn   hello_hello.6o2far8qyljljgwahds6pqrgb     hello-world:latest   node3     Ready           Ready less than a second ago
wgp9hf0ivajv    \_ hello_hello.6o2far8qyljljgwahds6pqrgb   hello-world:latest   node3     Shutdown        Complete 1 second ago
3274m4l4cf1d    \_ hello_hello.6o2far8qyljljgwahds6pqrgb   hello-world:latest   node3     Shutdown        Complete 9 seconds ago
c8lp7t71y9x8    \_ hello_hello.6o2far8qyljljgwahds6pqrgb   hello-world:latest   node3     Shutdown        Complete 15 seconds ago
68a30dpoer2i    \_ hello_hello.6o2far8qyljljgwahds6pqrgb   hello-world:latest   node3     Shutdown        Complete 22 seconds ago
r7cz5gaxghs8   hello_hello.kv0o3q4vy9cttsv66slml7q2n     hello-world:latest   node1     Ready           Ready 3 seconds ago
uipommkun2bz    \_ hello_hello.kv0o3q4vy9cttsv66slml7q2n   hello-world:latest   node1     Shutdown        Complete 4 seconds ago
iuw7w43mt34s    \_ hello_hello.kv0o3q4vy9cttsv66slml7q2n   hello-world:latest   node1     Shutdown        Complete 11 seconds ago
md9jqa42g20l    \_ hello_hello.kv0o3q4vy9cttsv66slml7q2n   hello-world:latest   node1     Shutdown        Complete 17 seconds ago
ntjsnf5aqd2t    \_ hello_hello.kv0o3q4vy9cttsv66slml7q2n   hello-world:latest   node1     Shutdown        Complete 23 seconds ago
vrwzcp2e75xy   hello_hello.vktrun1l2rvmov829r3pkq9fr     hello-world:latest   manager   Ready           Ready less than a second ago
t9dwf3qket1y    \_ hello_hello.vktrun1l2rvmov829r3pkq9fr   hello-world:latest   manager   Shutdown        Complete 2 seconds ago
ok3tbu0hd9zo    \_ hello_hello.vktrun1l2rvmov829r3pkq9fr   hello-world:latest   manager   Shutdown        Complete 9 seconds ago
rtsx0d3rbglq    \_ hello_hello.vktrun1l2rvmov829r3pkq9fr   hello-world:latest   manager   Shutdown        Complete 15 seconds ago
d5puswsuv948    \_ hello_hello.vktrun1l2rvmov829r3pkq9fr   hello-world:latest   manager   Shutdown        Complete 22 seconds ago
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$
```

Figure 2.8. En mode `global`, Swarm place une copie par nœud : `manager`, `node1`, `node2` et `node3`. Le nom de chaque copie contient l'identifiant de son nœud.

TD 3 : Traefik et Portainer

Dans ce TD, j'ai placé Traefik, un reverse proxy, devant mon cluster Swarm : chaque application répond à son propre nom de domaine au lieu d'un port. J'ai ensuite installé Portainer pour gérer le cluster depuis le navigateur.

Exercice 1 : Déployer Traefik

1.1 Relancer le cluster

J'ai repris le cluster du TD 2 en publiant le port 80 du manager, pour que le navigateur du PC puisse joindre Traefik.

mfp/swarm/compose.yml

```yaml
manager:
  image: docker:dind
  hostname: manager
  privileged: true
  ports:
    - "80:80" # Traefik, reachable from the host browser
```

Depuis mfp/swarm

```
docker compose up -d
docker compose exec manager docker node ls
```

Seul le manager a été recréé, et `node ls` affichait de nouveau les 4 nœuds `Ready`.

1.2 Déployer Traefik sur le manager

La stack contient Traefik, placé sur le manager, et `whoami`, une application de test qui se déclare à Traefik par ses labels.

mfp/swarm/traefik.compose.yml

```yaml
  # Deploy the Whoami application
  whoami:
    image: traefik/whoami
    networks:
      - web
    deploy:
      labels:
        - "traefik.enable=true"
        - "traefik.http.routers.whoami.rule=Host(`whoami.swarm.localhost`)"
        - "traefik.http.routers.whoami.entrypoints=web"
        - "traefik.http.services.whoami.loadbalancer.server.port=80"

# Define the overlay network for Swarm
networks:
  web:
    external: true
```

Le réseau `web` est déclaré `external` : je l'ai donc créé avant le déploiement, en mode overlay pour qu'il couvre tous les nœuds.

Commandes

```
docker compose exec manager docker network create --driver overlay web
docker compose exec manager docker network ls --filter name=web
```

J'ai ensuite supprimé la stack et le service du TD 2, puis j'ai déployé la stack Traefik.

Commandes

```
docker compose exec manager docker stack rm hello
docker compose exec manager docker service rm web
docker compose cp traefik.compose.yml manager:/home/manager/
docker compose exec manager docker stack deploy -c /home/manager/traefik.compose.yml traefik
docker compose exec manager docker service ls
```

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$ docker compose cp traefik.compose.yml manager:/home/manager/
 swarm-manager-1 Copying traefik.compose.yml to swarm-manager-1:/home/manager/
 swarm-manager-1 Copied traefik.compose.yml to swarm-manager-1:/home/manager/
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$ docker compose exec manager docker stack deploy -c /home/manage
r/traefik.compose.yml traefik
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating service traefik_traefik
Creating service traefik_whoami
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$ docker compose exec manager docker service ls
ID             NAME             MODE         REPLICAS   IMAGE                PORTS
34nw81r3tlo8   traefik_traefik   replicated   1/1        traefik:v3.6         *:80->80/tcp
mbp8wvpskz9n   traefik_whoami    replicated   1/1        traefik/whoami:latest
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$
```

Figure 3.1. Les services `traefik_traefik` et `traefik_whoami` sont à 1/1. Seul Traefik publie un port (80) ; whoami reste derrière lui.

1.3 Accéder à Traefik depuis le navigateur

Aucun vrai domaine ne pointe vers mon PC : j'ai donc ajouté les 2 adresses dans le fichier hosts de Windows, ouvert en administrateur.

C:\Windows\System32\drivers\etc\hosts

```
127.0.0.1   traefik.swarm.localhost
127.0.0.1   whoami.swarm.localhost
```

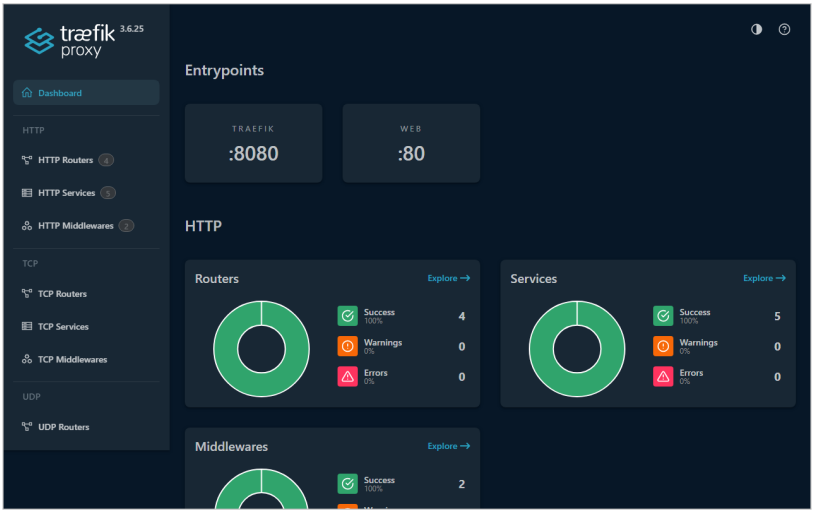


Figure 3.2. Le tableau de bord sur `traefik.swarm.localhost`. Le point d'entrée `web` écoute sur le port 80 et les 4 routeurs sont sans erreur.



Figure 3.3. La page `whoami.swarm.localhost`. L'en-tête `Host` et les en-têtes `X-Forwarded-*` ajoutés par Traefik montrent que la requête est passée par le reverse proxy.

Exercice 2 : Déployer une autre application

2.1 Tester l'application avec Docker Compose

L'application de vote compte 5 services : `vote`, `redis`, `worker`, `db` et `result`. Je l'ai d'abord lancée en local, avec le vote sur le port 8080 et les résultats sur le port 8081.

Depuis voting-app

```
docker compose up -d
docker compose ps --format 'table {{.Name}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}'
```

Problème rencontré : La première tentative s'est arrêtée sur `429 Too Many Requests`, car Docker Hub limite les téléchargements sans compte.

Solution : Je me suis connecté avec `docker login`, puis j'ai relancé `docker compose up -d`.

J'ai envoyé 5 votes avec `curl`, ce qui revient à cliquer sur les boutons du site.

Commandes

```
curl -X POST -d vote=a http://localhost:8080/   # Cats, 3 fois
curl -X POST -d vote=b http://localhost:8080/   # Dogs, 2 fois
```



Figure 3.4. Le site des résultats sur `localhost:8081` : 5 votes, 60 % pour Cats et 40 % pour Dogs.

Une fois le test terminé, j'ai arrêté l'application avec `docker compose down`.

2.2 Déployer l'application sur le cluster Swarm

La stack `docker-stack.yml` utilise des images déjà publiées, car Swarm ne construit pas d'images, et demande 2 répliques de `vote` et de `worker`. Je me suis connecté à Docker Hub dans le manager, et `--with-registry-auth` transmet cette connexion aux autres nœuds.

Depuis mfp/swarm

```
docker compose exec manager docker login
docker compose cp ../../voting-app/docker-stack.yml manager:/home/manager/
docker compose exec manager docker stack deploy --with-registry-auth -c
↪  /home/manager/docker-stack.yml voting
docker compose exec manager docker stack services voting
```

Au bout d'une minute environ, les 5 services étaient au complet, mais les ports 8080 et 8081 n'étaient publiés qu'à l'intérieur du cluster.

2.3 Publier le vote et les résultats avec Traefik

2.3.a Labels et redéploiement

Dans une nouvelle version de la stack, j'ai branché `vote` et `result` sur le réseau `web` et remplacé leurs ports par des labels Traefik. Voici la partie de `vote` ; celle de `result` est identique, avec `result.swarm.localhost`.

voting-app/docker-stack.traefik.yml

```yaml
networks:
  - frontend
  - web
deploy:
  replicas: 2
  labels:
    - "traefik.enable=true"
    - "traefik.http.routers.vote.rule=Host(`vote.swarm.localhost`)"
    - "traefik.http.routers.vote.entrypoints=web"
    - "traefik.http.services.vote.loadbalancer.server.port=80"
```

J'ai redéployé sous le même nom, `voting`, pour que Swarm mette à jour les 5 services existants.

Commandes

```
docker compose cp ../../voting-app/docker-stack.traefik.yml manager:/home/manager/
docker compose exec manager docker stack deploy --with-registry-auth -c
↪  /home/manager/docker-stack.traefik.yml voting
docker compose exec manager docker stack services voting
```

2.3.b Vérification

J'ai ajouté les 2 nouvelles adresses dans le fichier hosts.

hosts

```
127.0.0.1   vote.swarm.localhost
127.0.0.1   result.swarm.localhost
```

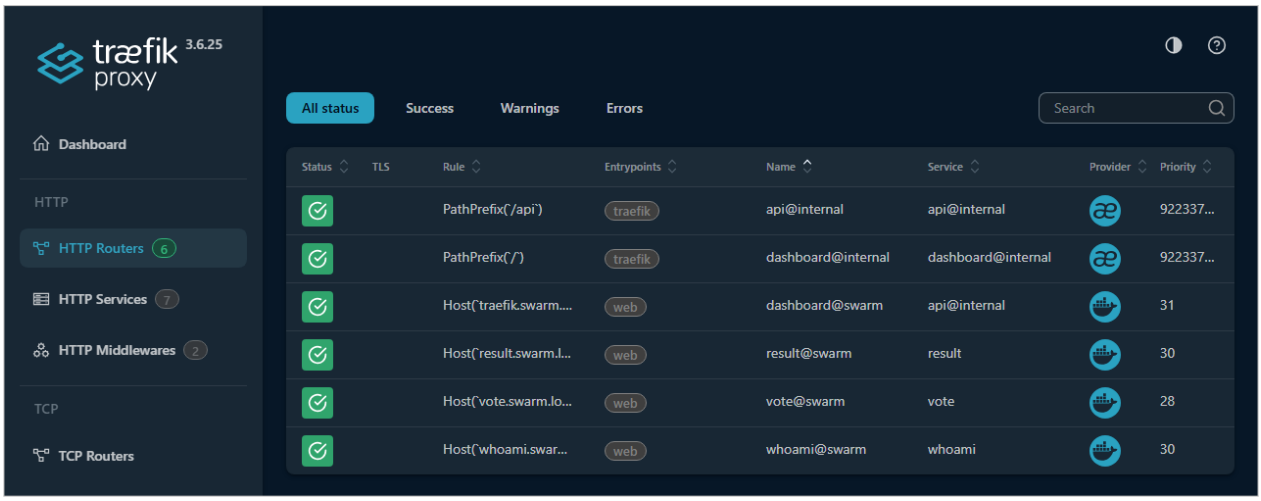


Figure 3.5. La page HTTP Routers de Traefik. Les routeurs `vote@swarm` et `result@swarm` ont été trouvés grâce aux labels et sont en succès.

J'ai de nouveau envoyé 5 votes, cette fois 2 pour Cats et 3 pour Dogs.

Commandes

```
curl -X POST -d vote=a http://vote.swarm.localhost/   # Cats, 2 fois
curl -X POST -d vote=b http://vote.swarm.localhost/   # Dogs, 3 fois
```



Figure 3.6. Le site des résultats sur `result.swarm.localhost` : 40 % pour Cats et 60 % pour Dogs. La base du cluster ne contient que les votes envoyés au cluster.

Exercice 3 : Déployer Portainer

3.1 Déployer Portainer sur le cluster

J'ai téléchargé la stack de Portainer pour Swarm : un agent sur chaque nœud (mode `global`) et l'interface sur le manager.

Depuis mfp/swarm

```
curl -sL https://downloads.portainer.io/ce-lts/portainer-agent-stack.yml -o
↪  portainer-agent-stack.yml
docker compose cp portainer-agent-stack.yml manager:/home/manager/
docker compose exec manager docker stack deploy --with-registry-auth -c
↪  /home/manager/portainer-agent-stack.yml portainer
docker compose exec manager docker stack services portainer
docker compose exec manager docker ps --filter name=portainer --format 'table
↪  {{.Names}}\t{{.Image}}\t{{.Status}}'
```

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$ curl -sL https://downloads.portainer.io/ce-lts/portainer-agent-stack.yml -o portainer-agent-stack.yml
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$ docker compose cp portainer-agent-stack.yml manager:/home/manager/
 swarm-manager-1 Copying portainer-agent-stack.yml to swarm-manager-1:/home/manager/
 swarm-manager-1 Copied portainer-agent-stack.yml to swarm-manager-1:/home/manager/
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$ docker compose exec manager docker stack deploy --with-registry-auth -c /home/manager/portainer-agent-stack.yml portainer
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network portainer_agent_network
Creating service portainer_agent
Creating service portainer_portainer
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$ docker compose exec manager docker stack services portainer
ID             NAME                MODE         REPLICAS   IMAGE                          PORTS
gdji5i54aezv   portainer_agent     global       4/4        portainer/agent:lts
ii02x8cee6at   portainer_portainer  replicated   1/1        portainer/portainer-ce:lts   *:8000->8000/tcp, *:9000->9000/tcp, *:9443->9443/tcp
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$ docker compose exec manager docker ps --filter name=portainer --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}'
NAMES                                                IMAGE                          STATUS
portainer_portainer.1.h75210yk9g7i719q6aj0335qn      portainer/portainer-ce:lts   Up 21 seconds
portainer_agent.vktrun1l2rvmov829r3pkq9fr.isijtnpif08a6yt1szfrqde8p   portainer/agent:lts          Up 23 seconds
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$
```

Figure 3.7. `portainer_agent` est à 4/4, un agent par nœud, et `portainer_portainer` est à 1/1. Les 2 conteneurs tournent bien sur le manager.

3.2 Publier Portainer avec Traefik

Comme pour `whoami`, j'ai branché Portainer sur le réseau `web` et ajouté ses labels, vers son port 9000.

mfp/swarm/portainer-agent-stack.traefik.yml

```yaml
    networks:
      - agent_network
      - web
    deploy:
      mode: replicated
      replicas: 1
      placement:
        constraints: [node.role == manager]
      labels:
        - "traefik.enable=true"
        - "traefik.http.routers.portainer.rule=Host(`portainer.swarm.localhost`)"
        - "traefik.http.routers.portainer.entrypoints=web"
        - "traefik.http.services.portainer.loadbalancer.server.port=9000"

networks:
  agent_network:
    driver: overlay
```

```
    attachable: true
  web:
    external: true
```

J'ai redéployé sous le même nom de stack, ce qui a mis à jour les 2 services sans les recréer.

Commandes

```
docker compose cp portainer-agent-stack.traefik.yml manager:/home/manager/
docker compose exec manager docker stack deploy --with-registry-auth -c
↪  /home/manager/portainer-agent-stack.traefik.yml portainer
docker compose exec manager docker stack services portainer
```

3.3 Ouvrir Portainer dans le navigateur

3.3.a Setup token

J'ai ajouté l'adresse de Portainer dans le fichier hosts.

hosts

```
127.0.0.1   portainer.swarm.localhost
```

Au premier lancement, Portainer demande un setup token, que j'ai trouvé dans ses logs.

Commandes

```
docker compose exec manager docker service logs --raw portainer_portainer 2>&1 | grep -A2
↪  setup_token=
```

3.3.b Création du compte

Sur `http://portainer.swarm.localhost`, j'ai créé le compte `admin` avec ce token et un mot de passe d'au moins 12 caractères. Portainer ne laisse que 5 minutes après son démarrage pour le faire.

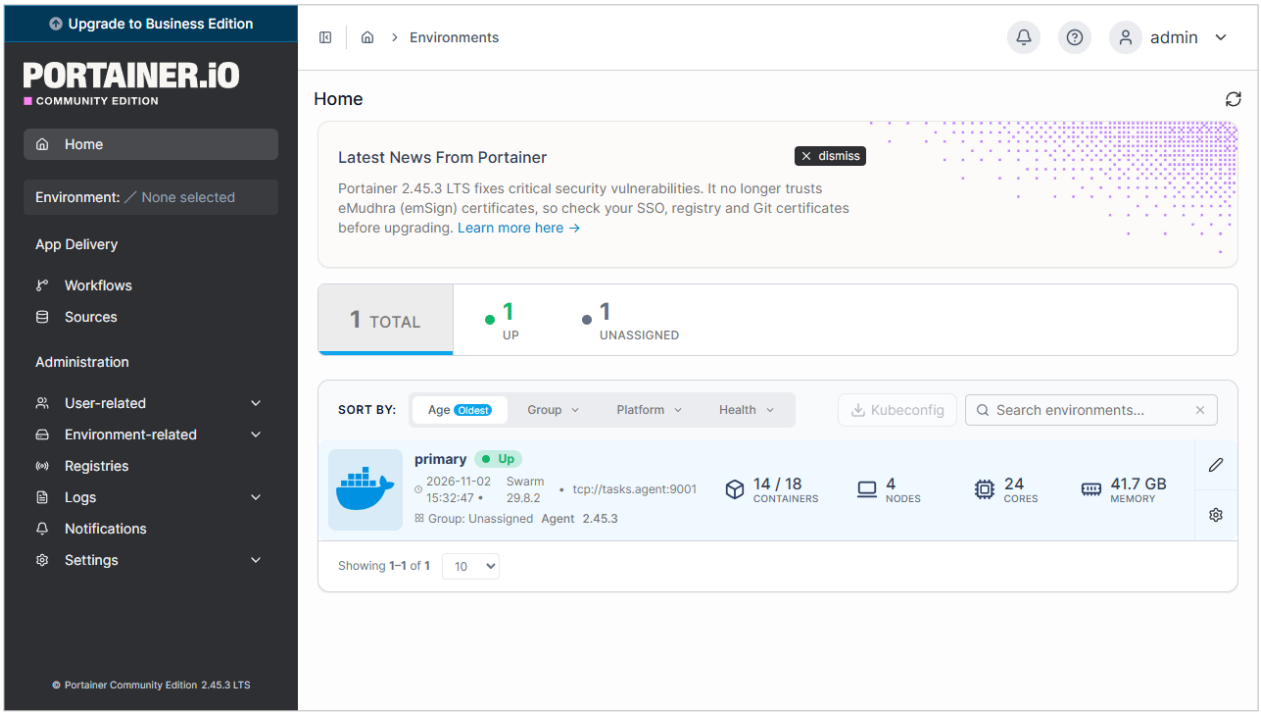


Figure 3.8. L'accueil de Portainer. L'environnement `primary` est `Up`, de type Swarm, avec les 4 nœuds trouvés par les agents.

Exercice 4 : Redéployer depuis Portainer

4.1 Supprimer la stack depuis le manager

J'ai supprimé la stack de l'application de vote, créée en ligne de commande.

Depuis mfp/swarm

```
docker compose exec manager docker stack ls
docker compose exec manager docker stack rm voting
docker compose exec manager docker stack ls
```

`stack rm` a supprimé les 5 services et les 2 réseaux de `voting` : il ne restait plus que `portainer` et `traefik`.

4.2 Republier la stack depuis Portainer

Dans Portainer, j'ai ouvert `primary`, puis « Stacks » et « Add stack ». J'ai nommé la stack `voting`, envoyé le fichier `docker-stack.traefik.yml` de la question 2.3 avec la méthode « Upload », puis cliqué sur « Deploy the stack ».

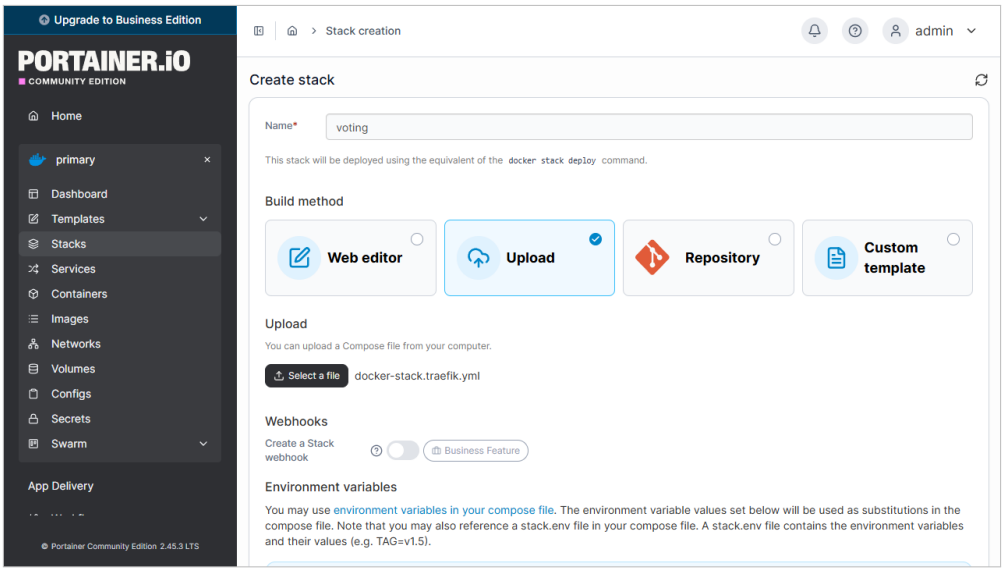


Figure 3.9. Le formulaire de création de la stack `voting`. Portainer fait l'équivalent d'un `docker stack deploy`.

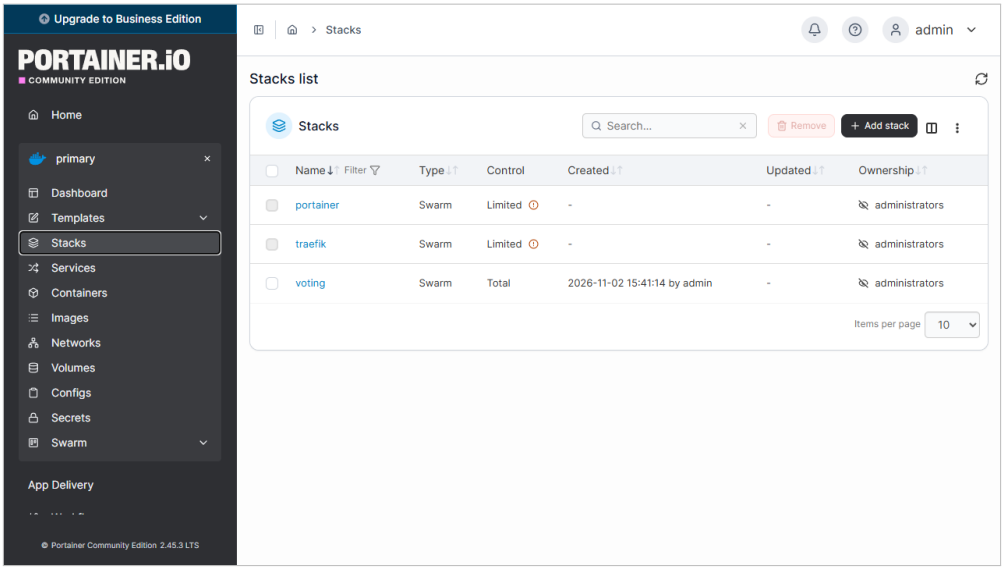


Figure 3.10. `voting` est de retour, avec un contrôle « Total » et créée par `admin`. Les stacks créées en ligne de commande restent « Limited » : Portainer ne peut pas modifier leur fichier.

4.3 Vérifier l'accès depuis le navigateur

Après ce redéploiement, le vote et les résultats répondaient de nouveau à leurs adresses : la stack déployée par Portainer fonctionne comme celle déployée en ligne de commande.

> **Problème rencontré :** La page des résultats affichait « No votes yet » : la base avait été recréée sur `node2` au lieu de `node3`, avec un volume neuf.
>
> **Solution :** Je n'ai rien corrigé : les anciens votes sont restés dans le volume de `node3`.

4.4 Déployer l'API de MFP depuis Portainer

4.4.a Création de la stack

Ma stack reprend la base et le serveur de MFP. Le serveur utilise l'image publiée sur GHCR et passe par Traefik sur `api.swarm.localhost`.

mfp/swarm/mfp-api.stack.yml

```yaml
  server:
    image: ghcr.io/dark-shad0w/clusterisation-conteneurs/server:latest
    networks:
      - backend
      - web
    deploy:
      labels:
        - "traefik.enable=true"
        - "traefik.http.routers.mfp-api.rule=Host(`api.swarm.localhost`)"
        - "traefik.http.routers.mfp-api.entrypoints=web"
        - "traefik.http.services.mfp-api.loadbalancer.server.port=3000"

networks:
  backend:
  # overlay network shared with Traefik (created in TD3 exercise 1)
  web:
    external: true
```

Dans Portainer, j'ai créé la stack `mfp` comme à la question 4.2, avec le fichier `mfp-api.stack.yml`.

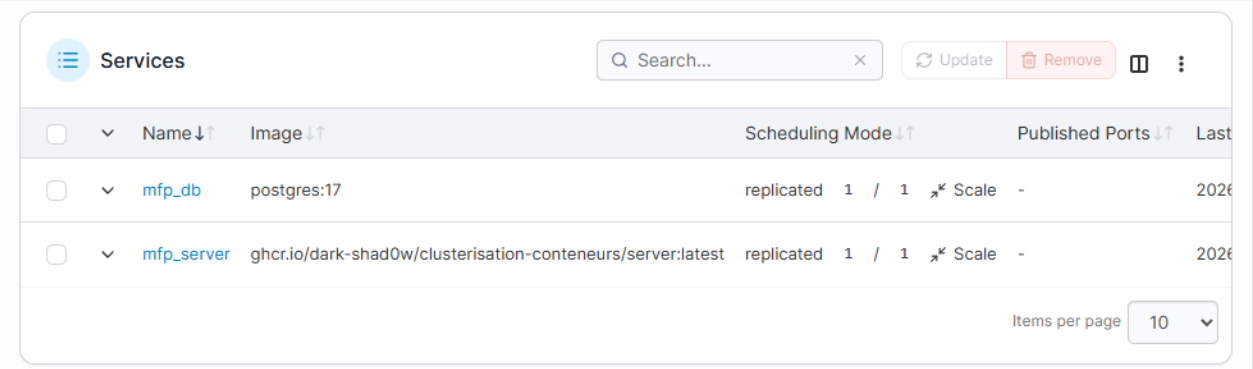


Figure 3.11. Les 2 services de la stack `mfp` sont au complet. `mfp_server` utilise l'image publiée sur GHCR.

Problème rencontré : Au premier démarrage, le serveur s'arrêtait sur `getaddrinfo ENOTFOUND db`, car Swarm lance la base et le serveur en même temps.

Solution : Je n'ai rien eu à faire : Swarm a relancé le serveur, qui s'est connecté à la base au deuxième essai.

4.4.b Test avec Bruno

J'ai ajouté l'adresse de l'API dans le fichier hosts.

hosts

```
127.0.0.1   api.swarm.localhost
```

Pour ne pas modifier ma collection Bruno, j'en ai fait une copie qui vise `http://api.swarm.localhost` au lieu de `http://localhost:3000`.

Depuis mfp

```
cp -r mfp-bruno-api-test-collection ~/mfp-bruno-swarm
sed -i 's#http://localhost:3000#http://api.swarm.localhost#' ~/mfp-bruno-swarm/*.yml
cd ~/mfp-bruno-swarm && bru run
```

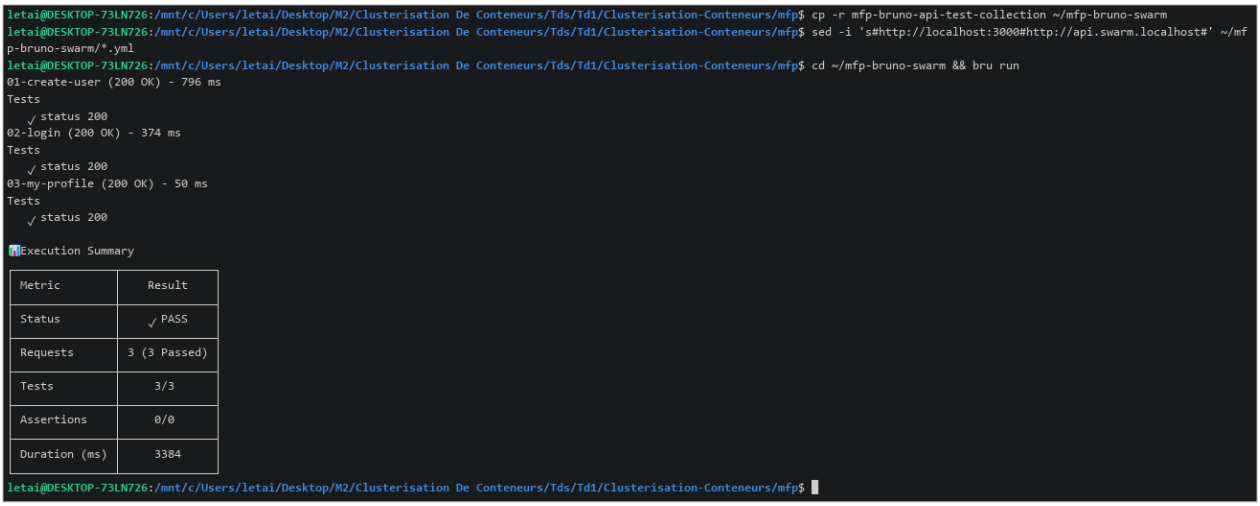


Figure 3.12. Les 3 requêtes répondent 200 OK et les 3 tests passent. L'API tourne sur le cluster et répond depuis le PC sur `api.swarm.localhost`.

TD 4 : Secrets et configs Docker Swarm

Swarm garde lui-même les secrets (informations sensibles, ni lisibles ni modifiables une fois créés) et les configs (fichiers lisibles mais non modifiables), et les monte comme des fichiers dans les conteneurs, quel que soit leur nœud. J'ai lancé toutes les commandes depuis le dossier `swarm/`.

Prérequis : relancer le cluster et passer en HTTPS

0.1 Relancer le cluster

J'ai publié le port 443 du manager, pour que Traefik réponde aussi en HTTPS.

swarm/compose.yml

```yaml
manager:
  image: docker:dind
  hostname: manager
  privileged: true
  ports:
    - "80:80" # Traefik, reachable from the host browser
    - "443:443" # Traefik HTTPS (TD4: Authelia only works over https)
  volumes:
    - /home/manager
```

Commandes

```
docker compose up -d
docker compose ps
docker compose exec manager docker node ls
```

Problème rencontré : Le manager a été recréé pour ajouter le port 443 et les conteneurs ont changé d'adresse IP : `node1`, `node2` et `node3` étaient `Down`.

Solution : J'ai fait quitter le swarm aux 4 nœuds, puis j'ai reconstruit le cluster avec un nouveau jeton.

Commandes

```
docker compose exec node1 docker swarm leave --force
docker compose exec node2 docker swarm leave --force
docker compose exec node3 docker swarm leave --force
docker compose exec manager docker swarm leave --force

docker compose exec manager docker swarm init
TOKEN=$(docker compose exec manager docker swarm join-token -q worker)
docker compose exec node1 docker swarm join --token $TOKEN manager:2377
docker compose exec node2 docker swarm join --token $TOKEN manager:2377
docker compose exec node3 docker swarm join --token $TOKEN manager:2377

docker compose exec manager docker node ls
```

Les 4 nœuds sont revenus `Ready`. Le réseau `web`, Traefik et Portainer ont disparu avec l'ancien cluster : je les ai redéployés plus loin.

0.2 Créer le certificat

Authelia refuse une adresse en `http://`. J'ai donc créé une autorité de certification (CA) locale et un certificat `*.swarm.localhost` signé par elle.

Commandes

```
mkdir -p certs
openssl req -x509 -newkey rsa:2048 -noenc -days 3650 -subj "/CN=Swarm Local CA" -keyout
↪  certs/ca.key -out certs/ca.crt
openssl req -newkey rsa:2048 -noenc -subj "/CN=*.swarm.localhost" -keyout
↪  certs/swarm.localhost.key -out certs/swarm.localhost.csr
openssl x509 -req -in certs/swarm.localhost.csr -CA certs/ca.crt -CAkey certs/ca.key
↪  -CAcreateserial -days 365 -extfile <(printf
↪  "subjectAltName=DNS:*.swarm.localhost,DNS:swarm.localhost") -out
↪  certs/swarm.localhost.crt
ls certs
openssl verify -CAfile certs/ca.crt certs/swarm.localhost.crt
```



Figure 4.1. La CA et le certificat sont créés dans `certs/`. `openssl verify` répond `OK` : le certificat est bien signé par ma CA.

J'ai ajouté la CA aux autorités de confiance de Windows, pour que le navigateur accepte le certificat.

Commandes

```
certutil.exe -user -addstore Root "$(wslpath -w certs/ca.crt)"
```

0.3 Le certificat dans le cluster : un secret et des configs

La clé privée va dans un secret. Le certificat et `traefik-tls.yml`, qui donne à Traefik son certificat par défaut, vont dans des configs.

swarm/traefik-tls.yml

```yaml
tls:
  stores:
    default:
      defaultCertificate:
        certFile: /certs/swarm.localhost.crt # config swarm_localhost_crt
        keyFile: /run/secrets/swarm.localhost.key # secret swarm_localhost_key
```

J'ai recréé le réseau `web`, puis les 2 configs et le secret.

Commandes

```
docker compose exec manager docker network create --driver overlay web
docker compose exec -T manager docker config create swarm_localhost_crt - <
↪  certs/swarm.localhost.crt
docker compose exec -T manager docker config create traefik_tls - < traefik-tls.yml
docker compose exec -T manager docker secret create swarm_localhost_key - <
↪  certs/swarm.localhost.key
docker compose exec manager docker config ls
docker compose exec manager docker secret ls
```

J'ai ensuite inspecté une config et un secret pour comparer.

Commandes

```
docker compose exec manager docker config inspect --pretty traefik_tls
docker compose exec manager docker secret inspect --pretty swarm_localhost_key
```

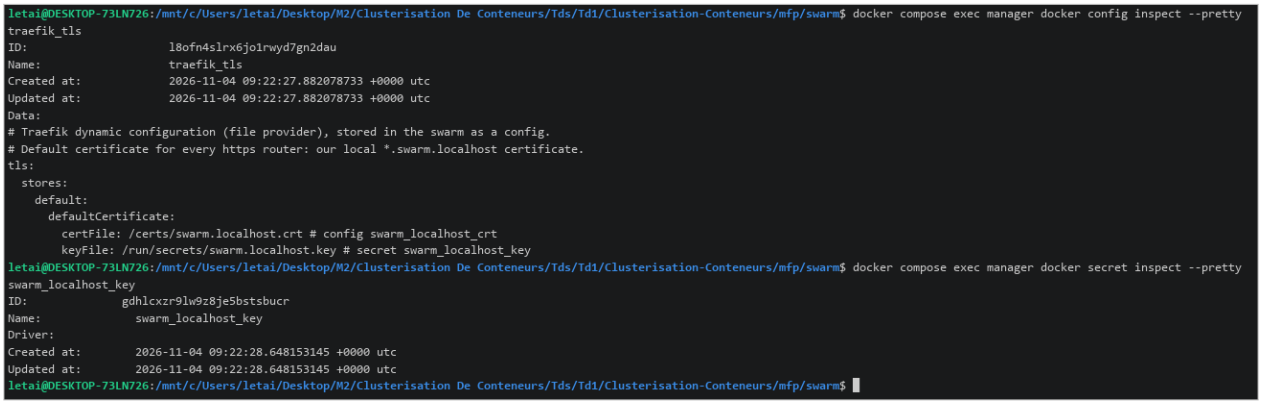


Figure 4.2. La config `traefik_tls` affiche son contenu (`Data`). Le secret `swarm_localhost_key` n'affiche que son nom et ses dates.

0.4 Traefik en HTTPS

La stack reprend celle du TD 3, avec un point d'entrée `websecure` sur le port 443. Les configs et le secret y arrivent sous forme de fichiers.

swarm/traefik-https.stack.yml

```yaml
services:
  traefik:
    networks:
      web:
        aliases:
          - authelia.swarm.localhost
    ports:
      - 80:80 # HTTP
      - 443:443 # HTTPS
    configs:
      - source: traefik_tls
        target: /etc/traefik/tls.yml
      - source: swarm_localhost_crt
        target: /certs/swarm.localhost.crt
    secrets:
      - source: swarm_localhost_key
```

```
      target: swarm.localhost.key # mounted in /run/secrets/
  command:
    - "--entrypoints.websecure.address=:443"
    - "--entrypoints.websecure.http.tls=true"
    - "--providers.file.filename=/etc/traefik/tls.yml"
  deploy:
    labels:
      - "traefik.http.routers.to-https.rule=HostRegexp(`.+`)"
      - "traefik.http.middlewares.to-https.redirectscheme.scheme=https"
```

Commandes

```
docker compose cp traefik-https.stack.yml manager:/home/manager/
docker compose exec manager docker stack deploy -c /home/manager/traefik-https.stack.yml
↪  traefik
docker compose exec manager docker service ls
```

J'ai testé depuis WSL, en donnant ma CA à `curl` avec `--cacert`.

Commandes

```
curl -s --cacert certs/ca.crt https://whoami.swarm.localhost | head -4
curl -sI http://dozzle.swarm.localhost | head -2
```

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$ curl -s --cacert certs/ca.crt https://whoami.swarm.localhos
t | head -4
Hostname: 13a6093ebbeb
IP: 127.0.0.1
IP: ::1
IP: 10.0.1.3
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$ curl -sI http://dozzle.swarm.localhost | head -2
HTTP/1.1 307 Temporary Redirect
Location: https://dozzle.swarm.localhost/
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$
```

Figure 4.3. `whoami` répond en HTTPS avec mon certificat. Une adresse sans routeur http, comme Dozzle, est redirigée vers `https://` par le routeur `to-https`.

0.5 Relancer Portainer et ajouter les adresses

J'ai redéployé la stack Portainer du TD 3 sans la modifier.

Commandes

```
docker compose exec manager docker volume ls
docker compose cp portainer-agent-stack.traefik.yml manager:/home/manager/
docker compose exec manager docker stack deploy -c
↪  /home/manager/portainer-agent-stack.traefik.yml portainer
docker compose exec manager docker service ls
```

Le volume `portainer_portainer_data` est resté sur le manager : Portainer a retrouvé son compte `admin`. J'ai ensuite ajouté les 3 adresses de l'exercice au fichier hosts de Windows.

hosts

```
127.0.0.1 dozzle.swarm.localhost
127.0.0.1 authelia.swarm.localhost
127.0.0.1 outline.swarm.localhost
```

Exercice 1 : Déployer des logiciels

1.1 Dozzle avec un utilisateur simple

Dozzle affiche en direct les logs des conteneurs. En mode Swarm, un Dozzle tourne sur chaque nœud et chacun montre les logs de tout le cluster.

1.1.a Le fichier users.yml

Dozzle lit ses utilisateurs dans `/data/users.yml` et génère lui-même l'empreinte du mot de passe.

Commandes

```
docker run --rm amir20/dozzle generate admin --password dozzle-admin-2026 --name "Admin"
```

1.1.b Le secret, créé dans Portainer

Comme le demande l'énoncé, j'ai créé ce secret dans Portainer (menu « Secrets », puis « Add secret »), en y collant le fichier généré.

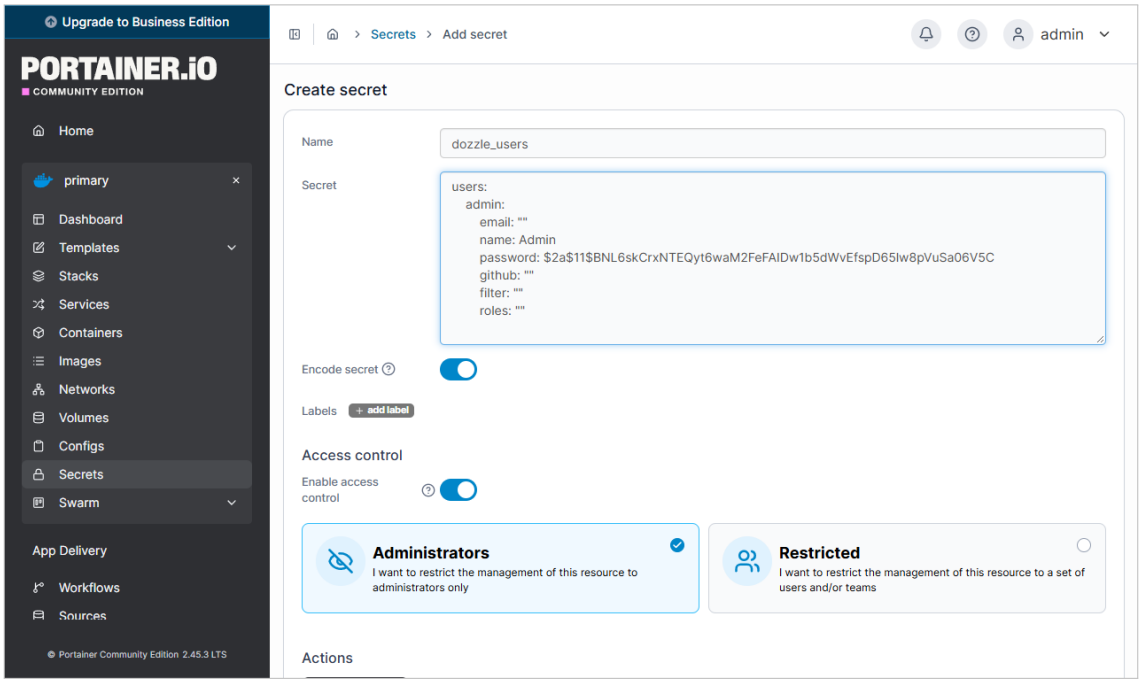


Figure 4.4. Création du secret `dozzle_users` dans Portainer. Le mot de passe n'y figure que sous forme d'empreinte bcrypt.

1.1.c La stack

Le secret est monté là où Dozzle cherche ses utilisateurs. Le mode `global` place une tâche sur chaque nœud.

swarm/dozzle.stack.yml

```
services:
  dozzle:
    image: amir20/dozzle:latest
    environment:
      DOZZLE_MODE: swarm # one Dozzle per node, they find each other on the "dozzle" network
      DOZZLE_AUTH_PROVIDER: simple # login form, users read from /data/users.yml
    volumes:
```

```
      - /var/run/docker.sock:/var/run/docker.sock
      - dozzle_data:/data
    secrets:
      - source: dozzle_users
        target: /data/users.yml # where Dozzle looks for its users
    networks:
      - dozzle
      - web
    deploy:
      mode: global # one task on each node, to read the logs of every node
      labels:
        - "traefik.http.routers.dozzle.entrypoints=websecure"
        # always send a browser to the same Dozzle task, so its login session stays valid
        - "traefik.http.services.dozzle.loadbalancer.sticky.cookie=true"

secrets:
  dozzle_users:
    external: true # created in Portainer
```

Commandes

```
docker compose cp dozzle.stack.yml manager:/home/manager/
docker compose exec manager docker stack deploy -c /home/manager/dozzle.stack.yml dozzle
docker compose exec manager docker service ps dozzle_dozzle
```

1.1.d Test dans le navigateur

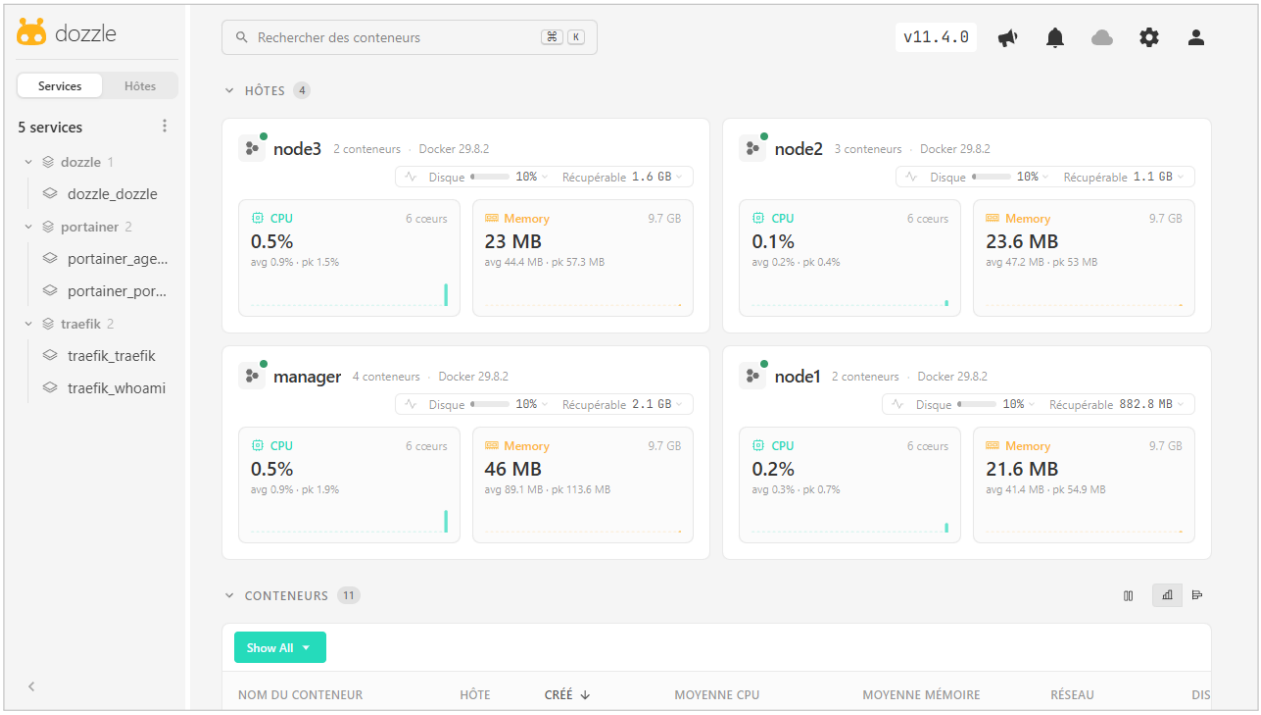


Figure 4.5. Connecté à `dozzle.swarm.localhost` avec le compte `admin`. Dozzle affiche les 4 nœuds du cluster et les services de mes stacks.

1.2 Authelia et connexion OpenID Connect

Avec OpenID Connect, Dozzle n'a plus de comptes : il envoie le navigateur sur Authelia, qui vérifie le mot de passe et lui renvoie un jeton signé.

1.2.a Les clés d'Authelia

J'ai généré 4 clés aléatoires et une clé RSA directement dans des secrets, sans les écrire sur le disque. J'ai aussi mis ma CA dans la config `swarm_ca`.

Commandes

```
for s in authelia_jwt_secret authelia_session_secret authelia_storage_key
↪  authelia_oidc_hmac; do openssl rand -hex 32 | docker compose exec -T manager docker
↪  secret create $s -; done
openssl genrsa 2048 | docker compose exec -T manager docker secret create authelia_oidc_key
↪  -
docker compose exec -T manager docker config create swarm_ca - < certs/ca.crt
docker compose exec manager docker secret ls
```

1.2.b Le compte dans un secret

J'ai rangé le compte dans `secrets/users_database.yml`, avec l'empreinte argon2 de son mot de passe générée par Authelia.

Commandes

```
docker run --rm authelia/authelia:4.39 authelia crypto hash generate argon2 --password
↪  'ahmed-authelia-2026'
cat secrets/users_database.yml
docker compose exec -T manager docker secret create authelia_users - <
↪  secrets/users_database.yml
```

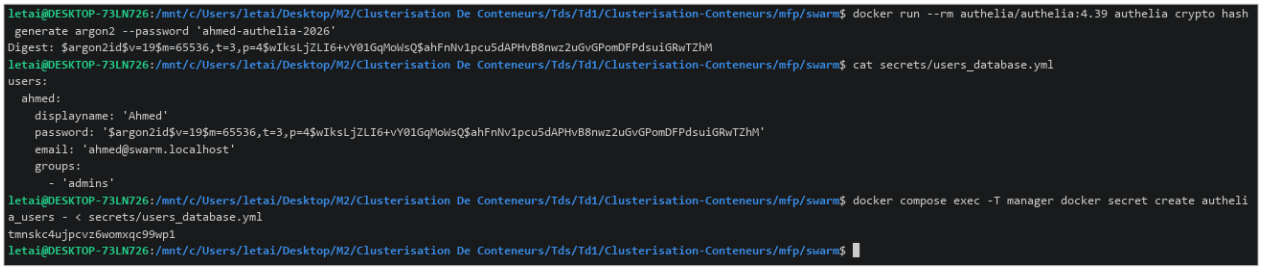


Figure 4.6. Le compte `ahmed`, membre du groupe `admins`, devient le secret `authelia_users`.

1.2.c Le client OIDC de Dozzle

Dozzle a besoin du client secret en clair : il va dans le secret `dozzle_oidc_secret`. Authelia n'en garde que l'empreinte.

Commandes

```
openssl rand -hex 36 | tr -d "\n" > secrets/dozzle_oidc_secret
docker run --rm authelia/authelia:4.39 authelia crypto hash generate pbkdf2 --variant
↪  sha512 --password "$(cat secrets/dozzle_oidc_secret)"
docker compose exec -T manager docker secret create dozzle_oidc_secret - <
↪  secrets/dozzle_oidc_secret
```

1.2.d La configuration dans une config

La configuration d'Authelia ne contient aucune valeur secrète : elle va donc dans une config. La clé OIDC y est lue dans son secret au démarrage.

swarm/authelia.configuration.yml

```yaml
# The accounts: the users_database.yml file is the swarm secret authelia_users
authentication_backend:
  file:
    path: '/config/users_database.yml'
    watch: false

session:
  cookies:
    - domain: 'swarm.localhost'
      authelia_url: 'https://authelia.swarm.localhost'

identity_providers:
  oidc:
    jwks:
      - key: {{ secret "/run/secrets/authelia_oidc_key" | mindent 10 "|" | msquote }}
    clients:
      - client_id: 'dozzle'
        client_name: 'Dozzle'
        # pbkdf2 digest of the secret stored in the swarm secret dozzle_oidc_secret
        client_secret: '$pbkdf2-sha512$310000$zQ.7CBa0mgiu8k8Zm3dTBA$cliYtETUut.EYaAb1C1x5G
        ↪ 0d8nJ7TBDq3ewmffU1bvgb/PbES0giHXdRx6Acqbgoa0V8yBz2Wk12VnIo7jdzLg'
        authorization_policy: 'one_factor'
        redirect_uris:
          - 'https://dozzle.swarm.localhost/api/auth/callback'
        token_endpoint_auth_method: 'client_secret_basic'
```

1.2.e Le déploiement

La config est montée en `/config/configuration.yml` et le compte en `/config/users_database.yml`. Les autres secrets arrivent dans `/run/secrets/`, lus par les variables `_FILE`.

swarm/authelia.stack.yml

```yaml
services:
  authelia:
    image: authelia/authelia:4.39
    environment:
      X_AUTHELIA_CONFIG: /config/configuration.yml
      X_AUTHELIA_CONFIG_FILTERS: template # enables {{ secret "..." }} in the configuration
      AUTHELIA_IDENTITY_VALIDATION_RESET_PASSWORD_JWT_SECRET_FILE:
      ↪ /run/secrets/authelia_jwt_secret
      AUTHELIA_SESSION_SECRET_FILE: /run/secrets/authelia_session_secret
      AUTHELIA_STORAGE_ENCRYPTION_KEY_FILE: /run/secrets/authelia_storage_key
      AUTHELIA_IDENTITY_PROVIDERS_OIDC_HMAC_SECRET_FILE: /run/secrets/authelia_oidc_hmac
    configs:
      - source: authelia_config
        target: /config/configuration.yml
    secrets:
      - source: authelia_users
        target: /config/users_database.yml # the account(s)
      - authelia_jwt_secret
      - authelia_session_secret
```

```
      - authelia_storage_key
      - authelia_oidc_hmac
      - authelia_oidc_key # private key that signs the OpenID Connect tokens
```

Commandes

```
docker compose exec -T manager docker config create authelia_config - <
↪  authelia.configuration.yml
docker compose cp authelia.stack.yml manager:/home/manager/
docker compose exec manager docker stack deploy -c /home/manager/authelia.stack.yml authelia
docker compose exec manager docker service ls --filter name=authelia
docker compose exec manager docker service logs --raw authelia_authelia 2>&1 | tail -4
```

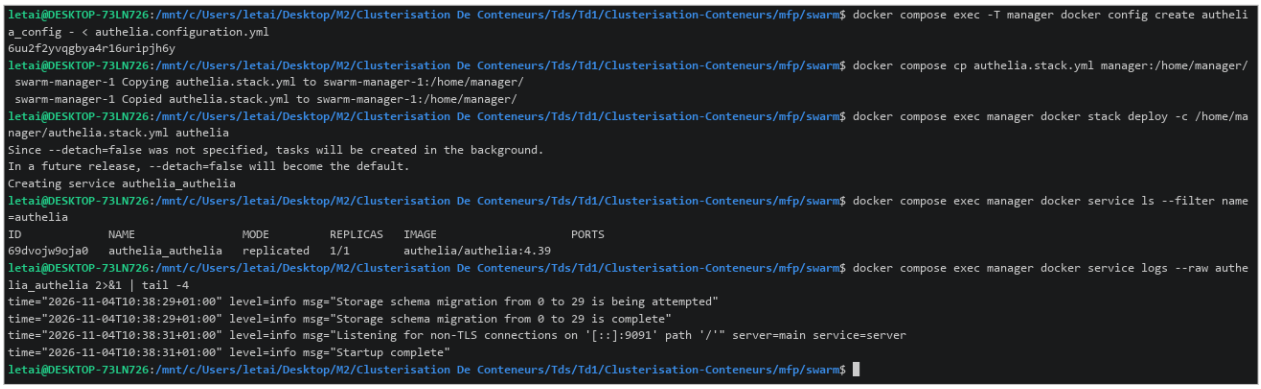


Figure 4.7. Le service `authelia_authelia` est à `1/1`. Ses logs se terminent par `Startup complete`.

1.2.f Brancher Dozzle sur Authelia

Dozzle appelle lui-même `https://authelia.swarm.localhost`. L'alias de Traefik sur le réseau `web` lui donne ce nom, et la config `swarm_ca` lui fait accepter mon certificat.

swarm/dozzle.oidc.stack.yml

```yaml
services:
  dozzle:
    image: amir20/dozzle:latest
    environment:
      DOZZLE_MODE: swarm
      DOZZLE_AUTH_PROVIDER: oidc # Authelia owns the accounts, users.yml is no longer read
      DOZZLE_AUTH_OIDC_ISSUER: https://authelia.swarm.localhost
      DOZZLE_AUTH_OIDC_CLIENT_ID: dozzle
      DOZZLE_AUTH_OIDC_CLIENT_SECRET_FILE: /run/secrets/dozzle_oidc_secret
      DOZZLE_AUTH_OIDC_NAME: Authelia # label of the login button
      # trust our local CA, to call https://authelia.swarm.localhost from the container
      SSL_CERT_FILE: /certs/ca.crt
    configs:
      - source: swarm_ca
        target: /certs/ca.crt
    secrets:
      - dozzle_oidc_secret # plain client secret (Authelia only keeps its digest)
```

Commandes

```
docker compose cp dozzle.oidc.stack.yml manager:/home/manager/
docker compose exec manager docker stack deploy -c /home/manager/dozzle.oidc.stack.yml
↪  dozzle
```

```
docker compose exec manager docker service ps dozzle_dozzle --filter desired-state=running
docker compose exec manager docker service logs --raw --since 2m dozzle_dozzle 2>&1 | grep
↪  -iE 'oidc|auth|error|fatal' | head -8
```

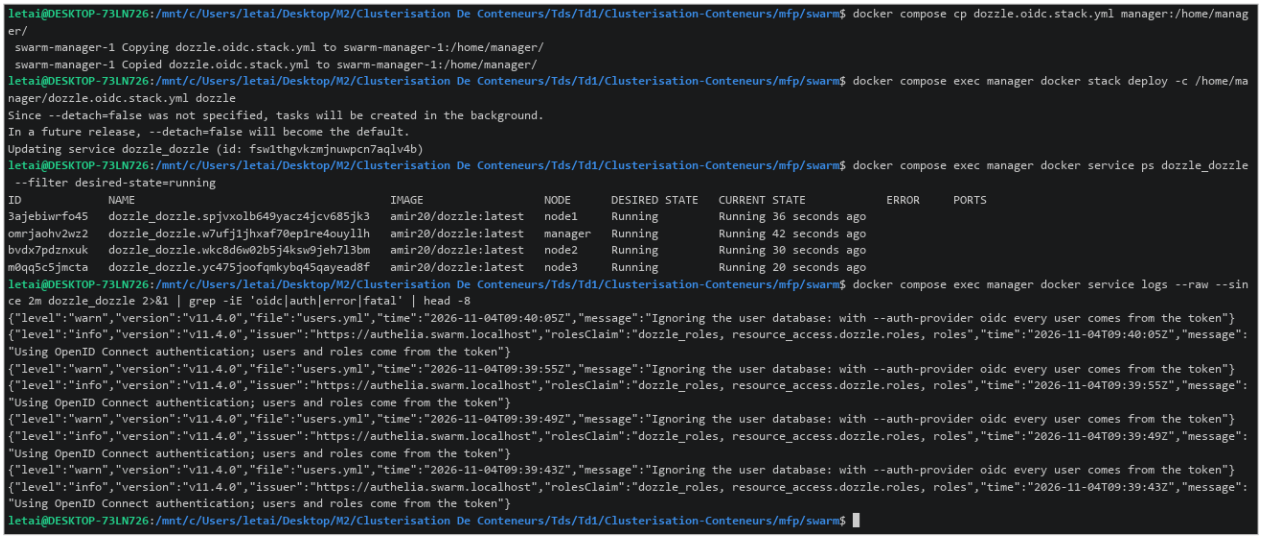


Figure 4.8. Le service Dozzle est mis à jour sur les 4 nœuds. Ses logs indiquent `Using OpenID Connect authentication`, avec Authelia comme émetteur.

1.2.g Rôles manquants et mise à jour de la config

Problème rencontré : Au retour d'Authelia, la connexion à Dozzle échouait avec `no roles claim found in the ID token`.

Solution : J'ai fait calculer par Authelia un attribut `dozzle_roles` à partir des groupes, envoyé dans le jeton quand Dozzle demande ce scope.

swarm/authelia.configuration.yml

```yaml
# Dozzle refuses a login without roles: members of the "admins" group get every role
definitions:
  user_attributes:
    dozzle_roles:
      expression: '"admins" in groups ? ["all"] : []'

identity_providers:
  oidc:
    # the dozzle_roles claim, sent in the ID token when the client asks for the dozzle_roles
    ↪  scope
    claims_policies:
      dozzle:
        id_token: ['dozzle_roles']
        custom_claims:
          dozzle_roles: {}
    scopes:
      dozzle_roles:
        claims: ['dozzle_roles']
    clients:
      - client_id: 'dozzle'
        # ...
        claims_policy: 'dozzle'
```

```
        scopes:
          # ...
          - 'dozzle_roles'
```

swarm/dozzle.oidc.stack.yml

```
DOZZLE_AUTH_OIDC_SCOPES: dozzle_roles # ask Authelia for the dozzle_roles claim
```

Une config ne se modifie pas : je l'ai supprimée puis recréée avec le nouveau fichier.

Commandes

```
docker compose exec manager docker config rm authelia_config
docker compose exec manager docker stack rm authelia
docker compose exec manager docker config rm authelia_config
docker compose exec -T manager docker config create authelia_config - <
↪  authelia.configuration.yml
docker compose exec manager docker stack deploy -c /home/manager/authelia.stack.yml authelia
docker compose exec manager docker service ls --filter name=authelia
```

Problème rencontré : Le premier `config rm` a été refusé, car la config était utilisée par le service `authelia_authelia`.

Solution : J'ai d'abord retiré la stack `authelia`, puis j'ai supprimé et recréé la config avant de redéployer.

J'ai aussi redéployé Dozzle, pour sa nouvelle variable.

Commandes

```
docker compose cp dozzle.oidc.stack.yml manager:/home/manager/
docker compose exec manager docker stack deploy -c /home/manager/dozzle.oidc.stack.yml
↪  dozzle
docker compose exec manager docker service ls --filter name=dozzle
```

1.2.h Résultat

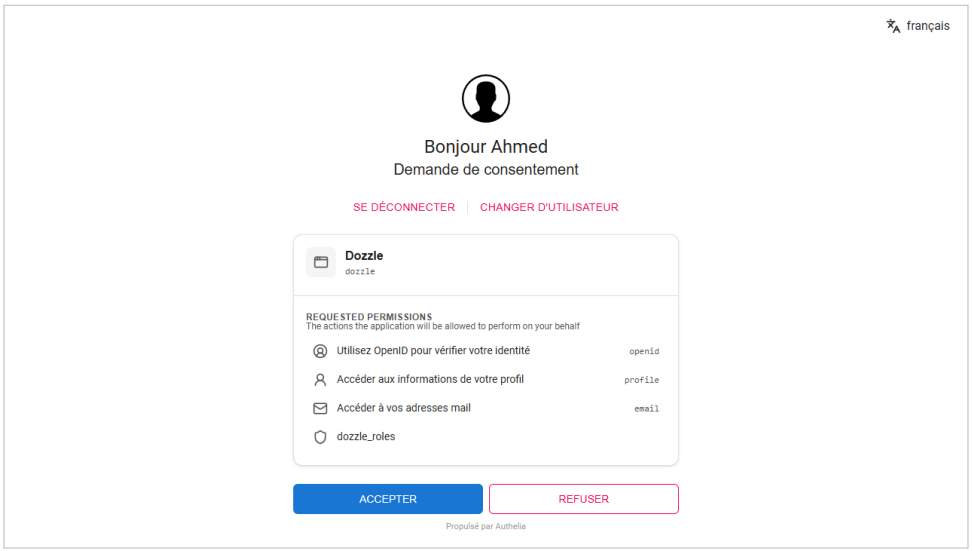


Figure 4.9. La demande de consentement d'Authelia contient maintenant le scope `dozzle_roles`.

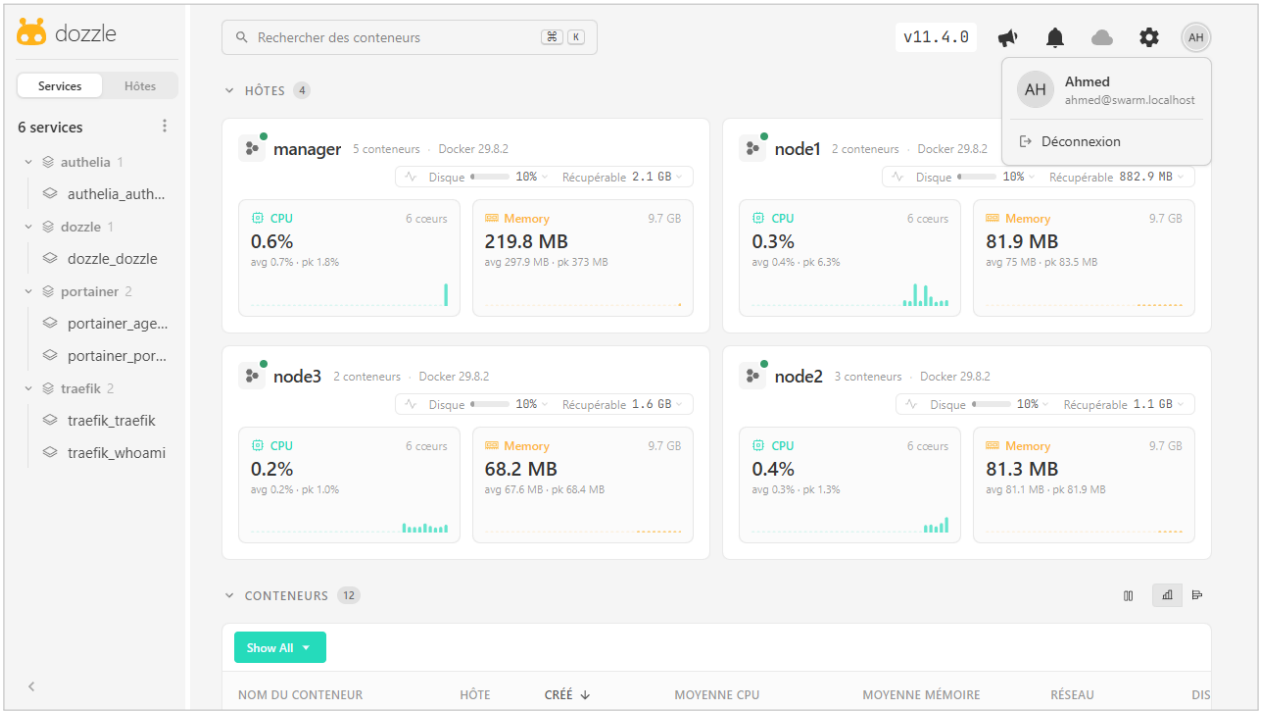


Figure 4.10. Je suis connecté à Dozzle en tant qu'Ahmed (`ahmed@swarm.localhost`), le compte du secret `authelia_users` vérifié par Authelia.

1.3 Bonus : Outline

Outline est un wiki d'équipe. Il utilise PostgreSQL et Redis, et se connecte à Authelia par OpenID Connect.

1.3.a Les secrets d'Outline

Outline lit lui aussi ses valeurs sensibles dans des variables `_FILE`. Le mot de passe de la base, tiré une seule fois dans `PW`, sert à 2 secrets.

Commandes

```
PW=$(openssl rand -hex 24)
printf %s "$PW" | docker compose exec -T manager docker secret create outline_db_password -
printf %s "postgres://outline:$PW@postgres:5432/outline" | docker compose exec -T manager
↪  docker secret create outline_database_url -
for s in outline_secret_key outline_utils_secret; do openssl rand -hex 32 | tr -d "\n" |
↪  docker compose exec -T manager docker secret create $s -; done
openssl rand -hex 36 | tr -d "\n" > secrets/outline_oidc_secret
docker run --rm authelia/authelia:4.39 authelia crypto hash generate pbkdf2 --variant
↪  sha512 --password "$(cat secrets/outline_oidc_secret)"
docker compose exec -T manager docker secret create outline_oidc_secret - <
↪  secrets/outline_oidc_secret
```

1.3.b Un nouveau client dans Authelia

J'ai ajouté le client `outline` à la configuration d'Authelia, avec l'empreinte de son client secret.

swarm/authelia.configuration.yml

```
# Bonus: Outline (TD4 ex. 1.3)
- client_id: 'outline'
  client_name: 'Outline'
  # pbkdf2 digest of the secret stored in the swarm secret outline_oidc_secret
  client_secret: '$pbkdf2-sha512$310000$wCiCZEIFdWY54dzdbN8SBQ$0Amgl90xzgb7n7V7CaYnJXa1A.l.
  ↪  kXpiReHdVhUukmDlbS27r1pejoF6AYE/oc3/FGQTa74loB8idUZSFWcBLw'
  public: false
  authorization_policy: 'one_factor'
  redirect_uris:
    - 'https://outline.swarm.localhost/auth/oidc.callback'
  scopes:
    - 'openid'
    - 'offline_access' # Outline uses a refresh token
    - 'profile'
    - 'email'
  response_types:
    - 'code'
  grant_types:
    - 'authorization_code'
    - 'refresh_token'
  token_endpoint_auth_method: 'client_secret_post'
```

Comme le rappelle l'énoncé, la config ne se met pas à jour : je l'ai supprimée puis recréée, cette fois après avoir retiré la stack.

Commandes

```
docker compose exec manager docker stack rm authelia
docker compose exec manager docker config rm authelia_config
docker compose exec -T manager docker config create authelia_config - <
↪  authelia.configuration.yml
docker compose exec manager docker stack deploy -c /home/manager/authelia.stack.yml authelia
docker compose exec manager docker service ls --filter name=authelia
```

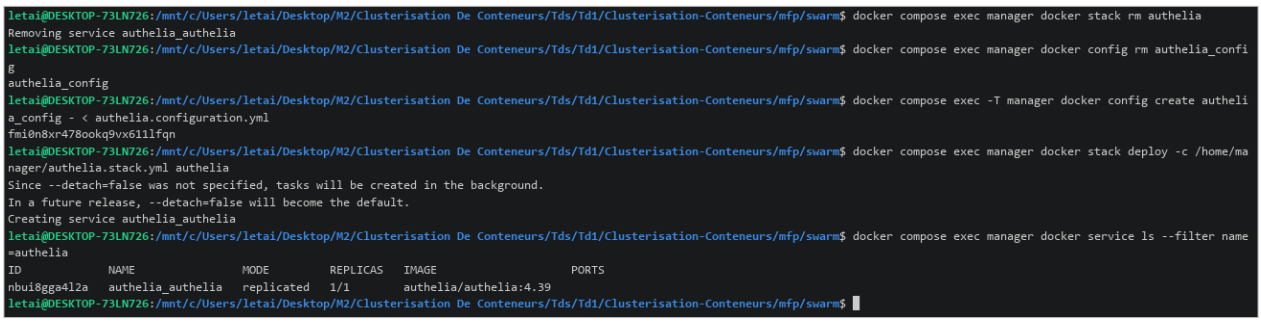


Figure 4.11. La stack étant retirée, `config rm` passe du premier coup. Authelia redémarre avec la config recréée.

1.3.c La stack

Toutes les valeurs sensibles passent par des variables `_FILE`. `NODE_EXTRA_CA_CERTS` fait accepter ma CA à Outline, comme `SSL_CERT_FILE` pour Dozzle.

swarm/outline.stack.yml

```yaml
services:
  outline:
    environment:
      SECRET_KEY_FILE: /run/secrets/outline_secret_key
      UTILS_SECRET_FILE: /run/secrets/outline_utils_secret
      DATABASE_URL_FILE: /run/secrets/outline_database_url
      # login with Authelia (client "outline" in authelia.configuration.yml)
      OIDC_CLIENT_ID: outline
      OIDC_CLIENT_SECRET_FILE: /run/secrets/outline_oidc_secret
      # trust our local CA, to call https://authelia.swarm.localhost from the container
      NODE_EXTRA_CA_CERTS: /certs/ca.crt
    configs:
      - source: swarm_ca
        target: /certs/ca.crt
    secrets:
      - outline_secret_key
      - outline_utils_secret
      - outline_database_url
      - outline_oidc_secret
  postgres:
    environment:
      POSTGRES_PASSWORD_FILE: /run/secrets/outline_db_password
```

Commandes

```
docker compose cp outline.stack.yml manager:/home/manager/
docker compose exec manager docker stack deploy -c /home/manager/outline.stack.yml outline
docker compose exec manager docker stack services outline
docker compose exec manager docker service logs --raw outline_outline 2>&1 | tail -4
```

1.3.d Résultat

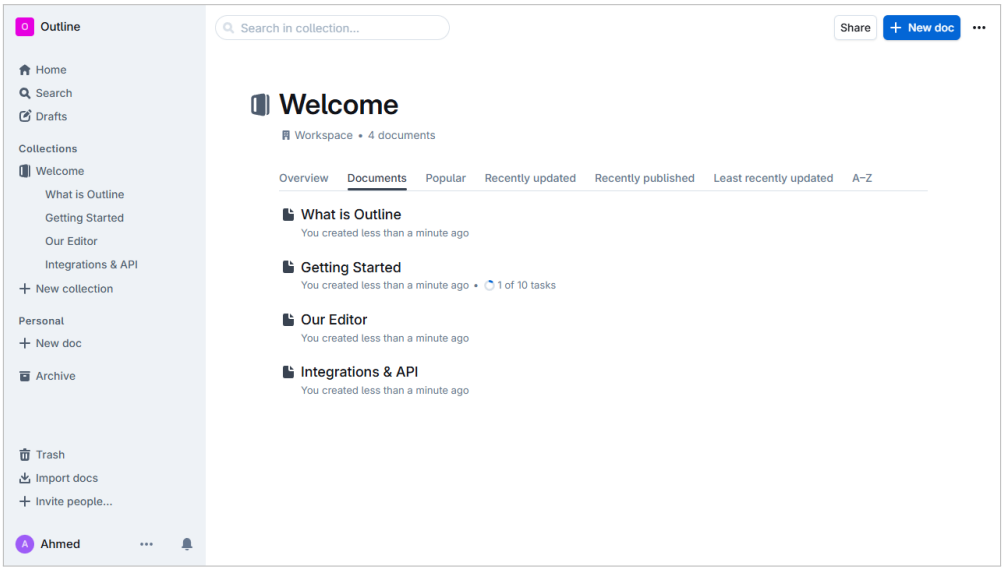


Figure 4.12. Connecté à Outline par Authelia, en tant qu'Ahmed.

Bilan

Les secrets et les configs créés pendant ce TD :

Nom	Type	Utilisé par	Créé depuis
`swarm_localhost_key`	secret	Traefik	ligne de commande
`swarm_localhost_crt`, `traefik_tls`	config	Traefik	ligne de commande
`swarm_ca`	config	Dozzle, Outline	ligne de commande
`dozzle_users`	secret	Dozzle (question 1.1)	Portainer
`dozzle_oidc_secret`	secret	Dozzle (question 1.2)	ligne de commande
`authelia_users`	secret	Authelia (le compte)	ligne de commande
`authelia_jwt_secret`, `authelia_session_secret`, `authelia_storage_key`, `authelia_oidc_hmac`, `authelia_oidc_key`	secret	Authelia	ligne de commande
`authelia_config`	config	Authelia	ligne de commande, recréée 2 fois
`outline_db_password`	secret	PostgreSQL d'Outline	ligne de commande
`outline_database_url`, `outline_secret_key`, `outline_utils_secret`, `outline_oidc_secret`	secret	Outline	ligne de commande

Une config se relit, un secret jamais, et aucun des deux ne se modifie. La même différence se voit dans Portainer.

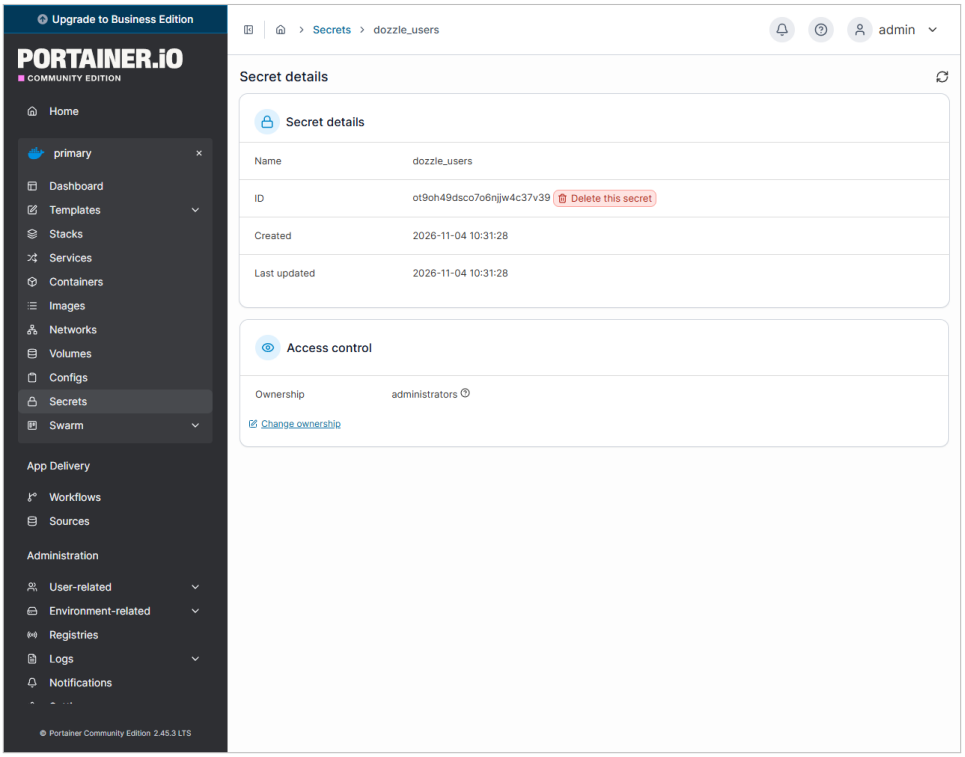


Figure 4.13. Dans Portainer, le secret `dozzle_users` n'affiche que son nom, son ID et ses dates. Le seul bouton est « Delete this secret ».

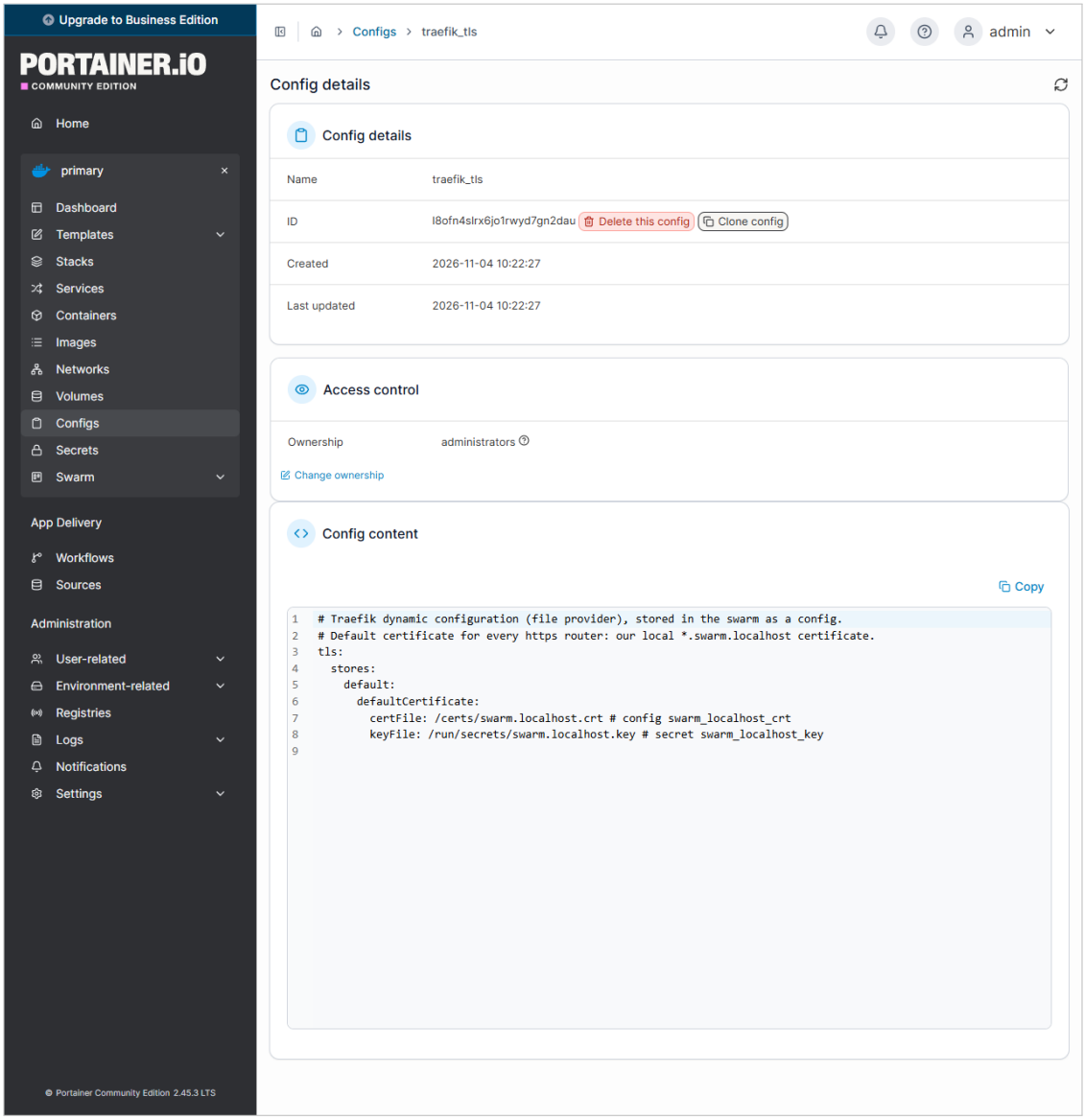


Figure 4.14. La config `traefik_tls` affiche son contenu. Pour la changer, Portainer ne propose que « Delete this config » ou « Clone config ».